

ACL: Account Linking in Online Social Networks With Robust Camera Fingerprint Matching

Liu Liu[✉], Xinwen Fu, *Senior Member, IEEE*, Xiaodong Chen, Jianpeng Wang, Zhongjie Ba[✉], *Member, IEEE*, Feng Lin[✉], *Senior Member, IEEE*, Li Lu[✉], *Member, IEEE*, and Kui Ren[✉], *Fellow, IEEE*

Abstract—Pseudonyms used in Online social networks (OSNs) post a great challenge to fighting against cyber crimes. To build a strong case, law enforcement may want to link multiple user accounts with pseudonyms to a physical suspect. Images based camera fingerprinting has been used for account linking when a suspect takes pictures and videos with his camera and posts them online. However, image post-processing software may introduce noise into images. This noise is hard to eliminate by conventional strategies, is partly resident in the estimated photo-response non-uniformity (PRNU) fingerprints, and interferes with matching fingerprints. We define this noise as software noise, which pollutes PRNU fingerprints and affects accounts linking in online social networks. In this article, we propose new approaches for camera fingerprint matching given software noise. The key idea is to determine the PRNU hardware noise correlation component with our new test statistic—fingerprint to software noise ratio (FITS). We performed extensive experiments and 10,000+ images taken by 90+ smartphones were used to validate our robust camera fingerprint matching system. The experiment results show FITS outperforms the state-of-the-art approaches for polluted fingerprints. This is the first work studying camera fingerprint matching with the presence of software noise.

Index Terms—PRNU, camera fingerprint, matching, user linking.

I. INTRODUCTION

CYBER criminals may abuse social networks (OSNs) to disseminate illegal contents such as child pornography images and videos, which were shared on Facebook, Kik, and Twitter [1], [2], [3], [4], [5]. Child pornography inflicts a severe long-term damage on the victims. Distributing and possessing child pornography is a serious federal crime in the United States.

Received 1 November 2023; revised 26 September 2024; accepted 19 December 2024. Date of publication 26 December 2024; date of current version 15 May 2025. This work was supported in part by the National Key R&D Program of China under Grant 2023YFB2904000 and Grant 2023YFB2904001, in part by the National Natural Science Foundation of China under Grant 62172359, Grant 61972348, and Grant 62102354, in part by the Fundamental Research Funds for the Central Universities under Grant 2021FZZX001-27, and in part by the Zhejiang Provincial Natural Science Foundation of China under Grant LD24F020010. (Corresponding author: Zhongjie Ba.)

Liu Liu, Xiaodong Chen, Jianpeng Wang, Zhongjie Ba, Feng Lin, Li Lu, and Kui Ren are with the Zhejiang University, Hangzhou, Zhejiang 310000, China, and also with the Jiaxing Research Institute, Zhejiang University, Jiaxing, Zhejiang 314031, China (e-mail: evence@zju.edu.cn; 22121164@zju.edu.cn; 12121132@zju.edu.cn; zhongjieba@zju.edu.cn; flin@zju.edu.cn; li.lu@zju.edu.cn; kuiren@zju.edu.cn).

Xinwen Fu is with the University of Massachusetts Lowell, Lowell, MA 01850 USA (e-mail: xinwen_fu@uml.edu).

Digital Object Identifier 10.1109/TDSC.2024.3522117

Under 18 U.S.C. §2251, a person who is found guilty of creating child pornography for the first time will receive fines and face 15–30 years prison time. The Office of Juvenile Justice and Delinquency Prevention (OJJDP) of the United States Department of Justice created the Internet Crimes Against Children (ICAC Task Force) in 1998 to fight online crimes against children.

Pseudonyms used for OSNs post a great challenge to fighting against cyber crimes. To hide their true identities, cyber criminals may create multiple online accounts and use fake names and information for these accounts [1], [2]. To build a strong case, law enforcement wants to collect as much evidence as possible about a crime. For example, law enforcement may want to link multiple OSN accounts to a suspect. Technologies such as PhotoDNA [6] cannot address this challenge. PhotoDNA creates a databases of hashes of known illegal images and detects child exploitation materials by matching hashes of suspicious images with the database. It cannot be used for account linking.

Device fingerprinting can be used for suspect account linking. Device fingerprints can be classified into two categories: Software and hardware. Software fingerprinting includes identifying factors such as browser fingerprint [7], IMEI code and MAC address [8] to establish a unique identifier for the device. However, such fingerprints can be easily manipulated. By contrast, hardware fingerprinting relies on manufacturing variations of hardware, whose physical uniqueness makes it more difficult to forge or alter. Previous studies on hardware components have been conducted, such as MEMS (Micro-Electro-Mechanical Systems) components [9], [10], acoustic components [11], [12], [13], CPU [14], GPU [15], and camera [16], [17].

The Photo-Response Non-Uniformity (PRNU) is a type of reliable camera fingerprint intrinsically embedded in images [18]. Devices such as mobile phones, tablets and laptops for OSN use are often equipped with camera modules, which makes it possible to use camera fingerprinting for account linking on OSNs [19], [20], [21]. A camera's image sensor consists of millions of photosensitive pixels which can exhibit non-uniformity in their responsiveness towards light due to imperfections in sensor manufacturing processes and silicon wafers. These tiny variations lead to uneven responses for each pixel across the sensor, and cause slight variations in pixel response even under identical lighting conditions. Such non-uniformity is modulated as a noise-like pattern and can be estimated from a target image through a denoising filter as introduced in Section II. When two PRNU fingerprints have been estimated, a test statistic such as the Peak to Correlation (PCE) can be calculated as the cross

correlation. If the test statistic is large, these PRNU fingerprints are correlated, which means two PRNU fingerprints are probably from the same device. On the contrary, if the test statistic approaches zero, indicating these two PRNU fingerprints are independent, two PRNU fingerprints are usually not from the same device. As a result, a threshold can be defined to determine whether two camera fingerprints are from the same device.

We find that post-processing techniques are often used to enhance images and videos. Meanwhile, these techniques bring software-induced noise into images and videos and affect the matching of camera fingerprints. This software-induced noise resides in the estimated PRNU camera fingerprint and cannot be effectively eliminated through denoising filtering and fingerprint enhancement methods. Hence, in the process of PRNU estimation, the camera fingerprint not only contains the desired hardware fingerprint but also includes additional software noise. In the process of matching, the test statistic involves both software noise correlation and hardware noise correlation. For cameras of the same model or brand, the post-processing software is often shared. Consequently, the software noises are highly correlated with the camera fingerprints of such cameras. Even when these camera fingerprints are from different devices, the test statistic no longer tends to approach zero statistically, which impacts matching accuracy and threshold setting. Unfortunately, there is no specific method designed for matching camera fingerprints polluted by software noise.

In this paper, we propose a robust camera fingerprint matching approach based on *fingerprint to software noise ratio* (FITS) to match camera fingerprints polluted by software noise. The core idea is to determine whether the PRNU related hardware noise correlation component exists in the test statistic through FITS. The new approach has two steps: (i) Estimating software noise correlation. (ii) Alleviating the influence. We first employ a test statistic to calculate the correlation between images captured by different devices of the same model. Since these images stemmed from distinct cameras, the correlation associated with hardware noise tends to approach zero. Consequently, the correlation actually reflects the correlation of software-related noise. For better estimation, we can compute an average through many images from different cameras. We then calculate the test correlation between two target camera fingerprints through traditional test statistics. Then we calculate the ratio of the test correlation over the estimated software noise correlation from Step (i), i.e., the *fingerprint to software noise ratio*. If the ratio is larger than 1, two images are from the same device, because the test correlation includes both hardware noise and software noise correlation components. If the ratio tends towards 1, two images are from different devices, because the test correlation only contains software noise correlation component. Therefore, a threshold around 1 can be set for matching camera fingerprints polluted by software noise.

Our main contributions can be summarized as follows:

- To the best of our knowledge, this is the first work defining software noise and studying the matching of camera fingerprints polluted by software noise.

- Using FITS, ACL can choose a common threshold to differentiate between matching and non-matching camera fingerprints polluted by various software noise among different brands, but conventional test statistics cannot complete this.
- We conducted extensive experiments using over 90 devices to validate the effectiveness and robustness of the FITS algorithm.
- Based on FITS, we propose a novel account linking system and conduct specific experiments targeting Facebook to show the advantages of FITS.
- The FITS algorithm can be applied to any camera fingerprint matching application as a replacement for the existing matching module, e.g., fighting against child pornography [22], [23].

The rest of this paper is organized as follows. We start with the definition of PRNU-based camera fingerprint and conventional fingerprint matching in Section II. In Section III, we present the system model, assumptions and problem definition. Section IV shows the evidence of software noise and models the software noise. Section V covers the limitation of conventional test statistics and new test statistics proposed in this paper. Section VI conducts the performance evaluation. We review the related existing work on camera fingerprint in Section VII. Section VIII concludes this research.

II. BACKGROUND

In this section, we introduce conventional fingerprint extraction and matching techniques.

A. Fingerprint Extraction

We are interested in extracting PRNU based fingerprints from images. An image captured by a digital camera can be modeled as follows [24]:

$$\mathbf{I} = \mathbf{I}^{(0)} + \mathbf{I}^{(0)}\mathbf{K} + \mathbf{\Theta}, \quad (1)$$

where $\mathbf{I}$ represents the captured image. $\mathbf{I}^{(0)}$ is the actual optical view. $\mathbf{\Theta}$ includes other noise components, such as quantization noise. The camera fingerprint $\mathbf{K}$ behaves like a white Gaussian noise and can often be extracted by a denoising filter $F(\cdot)$ [25]. The extracted noise residual $\mathbf{W}$ is given in (2) [26],

$$\mathbf{W} = \mathbf{I} - F(\mathbf{I}) = \mathbf{I}^{(0)}\mathbf{K} + \mathbf{\Xi}, \quad (2)$$

where $\mathbf{\Xi}$ is the combination of other white Gaussian noise. To reduce the impact of noise component $\mathbf{\Xi}$, a Maximum Likelihood Estimator (MLE) is usually applied. A camera fingerprint can be estimated as $\hat{\mathbf{K}}$ from n_{mle} images [24],

$$\hat{\mathbf{K}} = \frac{\sum_{i=1}^{n_{mle}} \mathbf{W}_i \mathbf{I}_i}{\sum_{i=1}^{n_{mle}} \mathbf{I}_i^2}, \quad (3)$$

where $\mathbf{W}_i$ is the noise residual extracted from image $\mathbf{I}_i$. n_{mle} is the number of images used to estimate a camera fingerprint. The higher the n_{mle} , the more precise the fingerprint estimation.

B. Fingerprint Matching

$\mathbf{K}_Q$ is a fingerprint in query and $\mathbf{K}_R$ is a reference fingerprint of a device. Camera fingerprint matching between $\mathbf{K}_Q$ and $\mathbf{K}_R$ can be formulated as a two-channel hypothesis testing problem [27]:

$$\begin{aligned} H_0 \text{ (non-matching pair)} : & \quad \mathbf{K}_Q \neq \mathbf{K}_R \\ H_1 \text{ (matching pair)} : & \quad \mathbf{K}_Q = \mathbf{K}_R, \end{aligned} \quad (4)$$

where H_0 is the null hypothesis (the two fingerprints are not matched) while H_1 is the alternative hypothesis (the two fingerprints are matched).

There are three popular fingerprint matching approaches [26], [28], including Cross-Correlation (CC), Normalized Cross-Correlation (NCC) and PCE,

$$CC(\mathbf{K}_Q, \mathbf{K}_R) = (\mathbf{K}_Q - \overline{\mathbf{K}_Q}) \odot (\mathbf{K}_R - \overline{\mathbf{K}_R}), \quad (5)$$

$$\begin{aligned} NCC(\mathbf{K}_Q, \mathbf{K}_R) &= \frac{CC(\mathbf{K}_Q, \mathbf{K}_R)}{\|\mathbf{K}_Q - \overline{\mathbf{K}_Q}\| \cdot \|\mathbf{K}_R - \overline{\mathbf{K}_R}\|} \\ &= \frac{CC(\mathbf{K}_Q, \mathbf{K}_R)}{\|\mathbf{K}_{Qm}\| \cdot \|\mathbf{K}_{Rm}\|}, \end{aligned} \quad (6)$$

$$PCE(\mathbf{K}_Q, \mathbf{K}_R) = \frac{NCC(\mathbf{K}_Q, \mathbf{K}_R, s_{\text{peak}})^2}{\frac{1}{mn-|\mathcal{N}|} \sum_{\mathbf{s}, \mathbf{s} \notin \mathcal{N}} NCC(\mathbf{K}_Q, \mathbf{K}_R, \mathbf{s})^2}, \quad (7)$$

where $\mathbf{K}_Q$ and $\mathbf{K}_R$ are two camera fingerprints. $\overline{\mathbf{X}}$ represents the mean value of $\mathbf{X}$. $\mathbf{X} \odot \mathbf{Y} = \sum_{i=1}^m \sum_{j=1}^n \mathbf{X}[i, j] \mathbf{Y}[i, j]$ indicates the dot product, and $\|\mathbf{X}\| = \sqrt{\mathbf{X} \odot \mathbf{X}}$ means the euclidean norm of $\mathbf{X}$. All arithmetic operations on matrices are element-wise. The dimension of camera fingerprints is $m \times n$. s_{peak} refers to the spatial shift at which the two-dimensional Normalized Cross-Correlation (NCC) achieves the peak value. $\mathcal{N}$ is a small neighborhood area around the peak, such as 11×11 pixels [24].

The signed PCE in (8) keeps the sign of NCC,

$$PCE_{\text{sgn}}(\mathbf{K}_Q, \mathbf{K}_R) = \frac{(NCC_{\text{peak}})^2 \cdot \text{sgn}(NCC_{\text{peak}})}{\frac{1}{mn-|\mathcal{N}|} \sum_{\mathbf{s}, \mathbf{s} \notin \mathcal{N}} NCC(\mathbf{K}_Q, \mathbf{K}_R, \mathbf{s})^2}, \quad (8)$$

where

$$NCC_{\text{peak}} = NCC(\mathbf{K}_Q, \mathbf{K}_R, s_{\text{peak}}). \quad (9)$$

$\text{sgn}(\cdot)$ returns the sign ($-1/+1$) of NCC.

In the approaches above, $CC(\cdot)$ has the lowest computational complexity, but its error rate could be relatively high. $NCC(\cdot)$ is a normalized $CC(\cdot)$. $PCE(\cdot)$ has the most robust estimation of the correlation but with the highest computational overhead. $PCE_{\text{sgn}}(\cdot)$ is often used in practice for accurate fingerprint matching.

C. Threshold Setting

To set a threshold for the test statistic in (5), (6) or (7) to reject the null hypothesis H_0 , we need to derive the distribution of the test statistic under H_0 in practice. For simplicity, we denote the

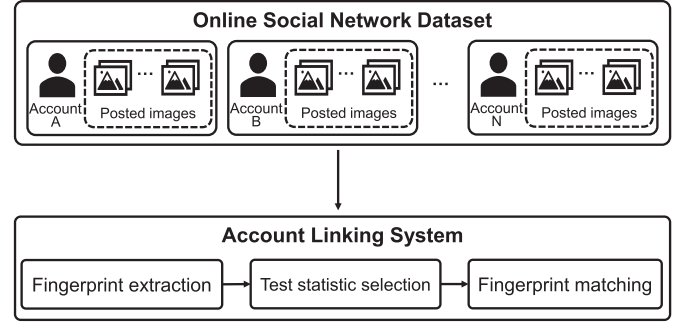


Fig. 1. Overview of the account linking system.

distribution of the test statistic under H_0 as H_0 distribution, and the distribution of the test statistic under H_1 as H_1 distribution. If the test statistic is bigger than the threshold, we can reject H_0 . In other words, it is probably that the two fingerprints are matched and the corresponding images are taken by the same camera.

III. SYSTEM MODEL AND PROBLEM DEFINITION

In this section, we first present the system model and the assumptions of the suspect account linking system based on camera fingerprints. We then define the problem that our system aims to address.

A. System Model

Fig. 1 illustrates our account linking system—ACL, which may be used to confirm that different online social network accounts actually belong to the same user through image based camera fingerprint matching. A user may create multiple accounts on OSNs with pseudonyms. The user posts illegal contents such as child pornography images and videos taken by his camera such as smartphone camera on those anonymous accounts. We want to confirm if the two suspicious accounts belong to the same user. The images and videos from suspect user accounts from the online social network dataset are the input of the ACL. Without loss of generality, we focus on image based account linking. Images can be extracted from videos and video based account linking can be performed similarly using the ACL.

The ACL takes three major steps for account linking. *Fingerprint extraction*: The system obtains images from two suspicious accounts from OSNs and extracts camera fingerprints from the images through a denoising filter, which is introduced in Section II-A. *Test statistic selection*: The system selects a suitable test statistic for fingerprint matching. For example, PCE can be chosen as the test statistic for better accuracy. CC is better for efficiency. *Fingerprint matching*: The system calculates the correlation between two camera fingerprints extracted from the images from the two accounts and compares it with a threshold. If the correlation is higher than the threshold, these fingerprints match and the two accounts are from the same user. Otherwise, these two accounts belong to different users.

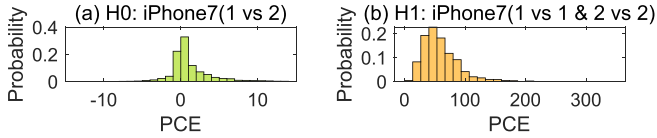


Fig. 2. H0 and H1 distributions of fingerprints from images taken by the default camera app of two iPhone 7. Each iPhone 7 takes 60 images.

B. Assumptions

We make the following assumptions about the suspect accounts and ACL.

- The ACL is trusted and free from attacks.
- The suspect posts images and videos created by the same camera of his device (e.g., smartphone) to different accounts.
- The suspect can remove the digital camera information data from the images and videos to hide his identity.
- The ACL has a large camera software dataset, which contains various camera fingerprints with software noise.

C. Problem Definition

We have observed that image post-processing techniques introduce software-induced noise into camera fingerprints. The account linking system based on camera fingerprints is no longer reliable with software noise. The test statistics including CC, NCC or PCE in Section II cannot work well.

Fingerprint matching with or without software noise

Input: Images/pictures that are obtained from two online social network accounts, are taken by the account owners and may go through post-processing

Question: How can we determine if those the two social network accounts belong to the same user by performing image-based camera fingerprint matching with or without the presence of software noise?

IV. SOFTWARE NOISE INTRODUCED BY IMAGE POST-PROCESSING

In this section, we first use experiment results to show the existence of software noise introduced by the camera post processing algorithms, then model images polluted by software noise and show how test statistics in Section II-B are affected.

A. Evidence of Software Noise

We first investigate the case of images without post-processing affecting camera fingerprinting. Without loss of generality, we often use the PCE to show the insights of fingerprint matching. Two PRNU fingerprints from two images taken by different cameras are not correlated. Therefore, the test statistic such as PCE should approach zero. That is, the PCE distribution under H_0 , i.e., the H_0 distribution introduced in Section II-C, shall have a mean around zero. Fig. 2 shows the H_0 and H_1 distributions for images taken by the default camera app of two iPhone 7 smartphones. The numbers 1 and 2 in the figure captions represent device numbers. “1 versus 2” indicates that the PCE results were computed from different devices. “1 versus

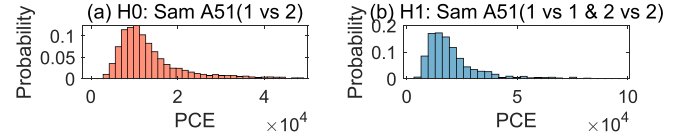


Fig. 3. H0 and H1 distributions about fingerprints of images captured by the built-in camera of Samsung A51. 60 images are captured for each device.

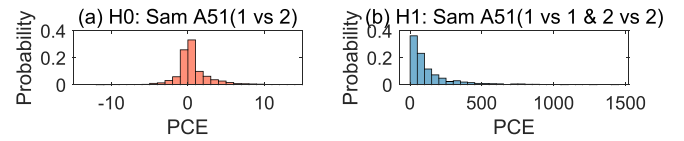


Fig. 4. H0 and H1 distributions about fingerprints of images captured by the B612 app of Samsung A51. 60 images are captured for each device.

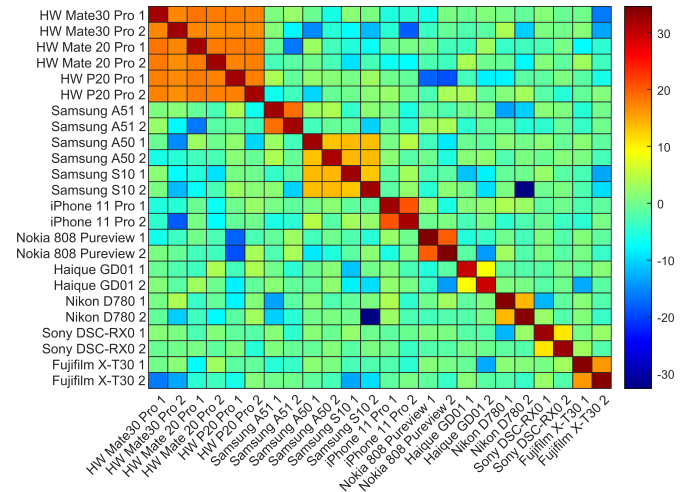


Fig. 5. Heat map of the PCE for smartphone cameras, surveillance camera (Haique GD01), and professional cameras incurring software noise.

1 & 2 versus 2” indicates that the PCE results were computed from the same device. It can be observed that for iPhone 7, the H_0 distribution has a mean of around zero. We observe the same phenomenon with other camera devices such as Samsung S8, VIVO IQOO U1X, and Mi 10 smartphones.

However, for images taken by the default camera app of two Samsung A51 smartphones, the H_0 distribution does not have a mean around zero any more as shown in Fig. 3. We then use a different popular third-party camera app (B612) on the two Samsung A51 smartphones to take images and draw the H_0 distribution again as shown in Fig. 4. The H_0 distribution has a mean around zero again. It can be observed from Figs. 3 and 4 that some post-processing software such as the default camera app of Samsung A51 introduces extra software noise, while others does not introduce this noise into images. We define software noise as the noise that is introduced by the post-processing software and pollutes the estimated camera fingerprint.

We perform extensive experiments to show that software noise is a widespread issue for various smartphone and camera models. Fig. 5 shows the heat map of the PCE for 24 cameras from twelve

device models with 2 devices from each model. All images are taken with the default camera app. Because of the wide range of the PCE, we calculate $\log(\text{PCE}^2)$ for the heat map. The redder the block color, the higher the PCE.

We make the following observations from Fig. 5: (i) For two different devices of the same model, the PCE still demonstrates a high value, ranging from 94.377 to 28148.597, which shows the evident correlation. According to [24], [29], [30], the PCE for non-matching fingerprint pairs is commonly under 60. Therefore, if the PCE for different devices of the same model is larger than 60, we say this model of device with a particular camera app produces software noise. (ii) It appears that similar post-processing algorithms may be used for different models from the same brand. For example, the PCE between Samsung A50 and S10 smartphones is large because of the software noise introduced by the similar post-processing software. (iii) For two devices from different brands, the PCE is very low. This shows software noises added by different post-processing algorithms used by different brands are independent.

B. Models With Software Noise

The post-processing software of a camera app may change an image and thus leave the software-related trace in the estimated camera fingerprint. We quantify such impact on the estimated camera fingerprint as software noise $\mathbf{S}$. The same camera app generates the same software noise on different devices since the imaging processing algorithm is the same. Based on our observations, we can model an image influenced by software noise as follows,

$$\mathbf{I} = \mathbf{I}^{(0)} + \mathbf{I}^{(0)}\mathbf{K} + \mathbf{I}^{(0)}\mathbf{S} + \boldsymbol{\Theta}, \quad (10)$$

When we use a denoising filter to extract the PRNU fingerprint, the extracted noise residual $\mathbf{W}$ is now given in (11),

$$\mathbf{W} = \mathbf{I} - F(\mathbf{I}). \quad (11)$$

A camera fingerprint polluted by software noise can be estimated as $\hat{\mathbf{K}}$ from n_{mle} samples of $\mathbf{W}$ through MLE in (3):

$$\hat{\mathbf{K}} = \alpha\mathbf{K} + \beta\mathbf{S} + \xi, \quad (12)$$

where ξ is the combination of other noises. Since we may not extract hardware noise and software noise completely, α and β are coefficients indicating the proportion of extracted fingerprints. Polluted fingerprints $\hat{\mathbf{K}}_Q$ and $\hat{\mathbf{K}}_R$ can be defined as:

$$\hat{\mathbf{K}}_Q = \alpha_Q\mathbf{K}_Q + \beta_Q\mathbf{S}_Q + \xi_Q \quad (13)$$

$$\hat{\mathbf{K}}_R = \alpha_R\mathbf{K}_R + \beta_R\mathbf{S}_R + \xi_R. \quad (14)$$

Since the PRNU hardware noise and software noise are produced by different mechanisms, they are independent. According to (5), for a pair of polluted fingerprints $\hat{\mathbf{K}}_Q$ and $\hat{\mathbf{K}}_R$, the cross correlation (CC) between them is now formulated as follows,

$$\begin{aligned} \text{CC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) \\ = \text{CC}(\alpha_Q\mathbf{K}_Q + \beta_Q\mathbf{S}_Q + \xi_Q, \alpha_R\mathbf{K}_R + \beta_R\mathbf{S}_R + \xi_R) \end{aligned}$$

$$\begin{aligned} &= (\alpha_Q\mathbf{K}_{Qm} + \beta_Q\mathbf{S}_{Qm} + \xi_{Qm}) \odot (\alpha_R\mathbf{K}_{Rm} + \beta_R\mathbf{S}_{Rm} + \xi_{Rm}) \\ &= \alpha_Q\alpha_R\mathbf{K}_{Qm} \odot \mathbf{K}_{Rm} + \beta_Q\beta_R\mathbf{S}_{Qm} \odot \mathbf{S}_{Rm} + \varphi_{QR} \\ &\approx \alpha_Q\alpha_R\mathbf{K}_{Qm} \odot \mathbf{K}_{Rm} + \beta_Q\beta_R\mathbf{S}_{Qm} \odot \mathbf{S}_{Rm}, \end{aligned} \quad (15)$$

where $\mathbf{X}_{Ym}$ refers to $(\mathbf{X}_Y - \overline{\mathbf{X}_Y})$. $\alpha_Q\alpha_R\mathbf{K}_{Qm} \odot \mathbf{K}_{Rm}$ and $\beta_Q\beta_R\mathbf{S}_{Qm} \odot \mathbf{S}_{Rm}$ are the hardware noise correlation and the software noise correlation, respectively. We use φ_{QR} to represent the sum of the other 7 noise components that are not matched. Specifically,

$$\begin{aligned} \varphi_{QR} &= \alpha_Q\beta_R\mathbf{K}_{Qm} \odot \mathbf{S}_{Rm} + \alpha_Q\mathbf{K}_{Qm} \odot \xi_{Rm} \\ &\quad + \beta_Q\alpha_R\mathbf{S}_{Qm} \odot \mathbf{K}_{Rm} + \beta_Q\mathbf{S}_{Qm} \odot \xi_{Rm} \\ &\quad + \alpha_R\xi_{Qm} \odot \mathbf{K}_{Rm} + \beta_R\xi_{Qm} \odot \mathbf{S}_{Rm} + \xi_{Qm} \odot \xi_{Rm}. \end{aligned} \quad (16)$$

Similarly, we can also derive the test statistic NCC in (6) and test statistic PCE in (7) for images polluted by software noise as follows,

$$\begin{aligned} \text{NCC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) &= \frac{\text{CC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R)}{\|\hat{\mathbf{K}}_{Qm}\| \cdot \|\hat{\mathbf{K}}_{Rm}\|} \\ &= r_{NCC}(\alpha_Q\alpha_R\mathbf{K}_{Qm} \odot \mathbf{K}_{Rm} + \beta_Q\beta_R\mathbf{S}_{Qm} \odot \mathbf{S}_{Rm} + \varphi_{QR}) \\ &\approx r_{NCC}(\alpha_Q\alpha_R\mathbf{K}_{Qm} \odot \mathbf{K}_{Rm} + \beta_Q\beta_R\mathbf{S}_{Qm} \odot \mathbf{S}_{Rm}), \end{aligned} \quad (17)$$

where

$$\begin{aligned} r_{NCC} &= \frac{1}{\|\hat{\mathbf{K}}_{Qm}\| \cdot \|\hat{\mathbf{K}}_{Rm}\|} \\ &= \frac{1}{\sqrt{\mathbf{K}_{Qm} \odot \mathbf{K}_{Qm}} \cdot \sqrt{\mathbf{K}_{Rm} \odot \mathbf{K}_{Rm}}}. \end{aligned} \quad (18)$$

$$\begin{aligned} \text{PCE}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) &= \frac{\text{NCC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R, \mathbf{s}_{\text{peak}})^2}{\frac{1}{mn-|\mathcal{N}|} \sum_{\mathbf{s}, \mathbf{s} \notin \mathcal{N}} \text{NCC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R, \mathbf{s})^2} \\ &= \frac{(\alpha_Q\alpha_R\mathbf{K}_{Qm} \odot \mathbf{K}_{Rm} + \beta_Q\beta_R\mathbf{S}_{Qm} \odot \mathbf{S}_{Rm} + \varphi_{QR})_{\mathbf{s}_{\text{peak}}}^2}{r_{PCE}} \\ &\approx \frac{(\alpha_Q\alpha_R\mathbf{K}_{Qm} \odot \mathbf{K}_{Rm} + \beta_Q\beta_R\mathbf{S}_{Qm} \odot \mathbf{S}_{Rm})_{\mathbf{s}_{\text{peak}}}^2}{r_{PCE}}, \end{aligned} \quad (19)$$

where $\mathbf{s}_{\text{peak}}$ is the spatial shift at which $\text{NCC}(\cdot)$ outputs the peak value, and

$$\begin{aligned} r_{PCE} &= \frac{\sum_{\mathbf{s}, \mathbf{s} \notin \mathcal{N}} (\alpha_Q\alpha_R\mathbf{K}_{Qm} \odot \mathbf{K}_{Rm_s} + \beta_Q\beta_R\mathbf{S}_{Qm} \odot \mathbf{S}_{Rm_s} + \varphi_{QR_s})^2}{mn - |\mathcal{N}|}. \end{aligned} \quad (20)$$

In the equations above, r_{NCC} and r_{PCE} are coefficients. CC, NCC and PCE all contain two parts: A hardware noise correlation and a software noise correlation. Because of the existence of software noise, it is hard to set a uniform threshold as before because the test statistics vary with device brands and

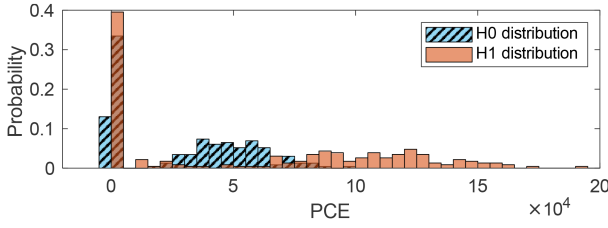


Fig. 6. Overlapping H0 and H1 distributions when fingerprints may or may not have software noise.

post-processing software. CC is also susceptible to the variation of the image resolution.

The mean of the H0 distribution can be used to infer if software noise exists. The mean of the term $\mathbf{K}_{Qm} \odot \mathbf{K}_{Rm}$ in (15), (17) and (19) is zero for unmatched camera fingerprints. If the H0 distribution has a mean of zero, the mean of $\mathbf{S}_{Qm} \odot \mathbf{S}_{Rm}$ in the three equations shall be zero, indicating the camera app does not produce software noise. Since the test statistic is the combination of the hardware noise correlation and the software noise correlation, if the H0 distribution has a non-zero mean, the camera app produces software noise.

V. FINGERPRINT MATCHING

In this section, we first show the drawbacks of conventional test statistics introduced in Section II-B, which fail to handle images polluted by software noise. Next we introduce our new test statistics given the existence of software noise and how to choose the threshold. At last we generalize the new test statistics so that they can be used for matching fingerprints blindly, disregarding the existence of software noise.

A. Limitation of Conventional Test Statistics

We now show the limitation of conventional test statistics under the interference of software noise component mentioned in Section IV-B with experiments. We choose different smartphones (Samsung A51, VIVO IQOO U1X, and Motorola Edge S) and digital cameras (Fujifilm X-T30 and Sony DSC-RX0) for the experiments. For smartphones, we choose the default camera apps to take images. Some devices (Samsung A51, Fujifilm X-T30 and Sony DSC-RX0) introduce heavy software noise but others (VIVO IQOO U1X and Motorola Edge S) do not. To demonstrate the diversity of potential software noise, we also intentionally introduce software noise into images taken by VIVO IQOO U1X by processing the images with a watermark algorithm DCT (Refer to Section VI-D) [31] to generate artificial software noise. Without loss of generality, we show the performance of PCE in (8) for camera fingerprint matching. Fig. 6 shows the H0 and H1 distributions for images taken by devices from different models and brands. We can see the H0 distribution has two segments, one for images without software noise on the left and the other for images with software noise on the right. The H1 distribution has a similar pattern. Because of the existence of software noise, the H0 and H1 distributions overlap each other very much. Therefore, the conventional test

statistics cannot be used any more for fingerprint matching in this context.

B. Alleviating Software Noise

Since the test statistic $\text{Cor}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R)$ is affected by the same software noise component, we want to estimate the correlation of software noise and alleviate it.

Estimating software noise correlation: Assume our account linking system knows every account's smartphone model and camera software, it can derive a set of camera fingerprints, $Z = \{\hat{\mathbf{K}}_{Z_i} | i = 1, \dots, n_Z\}$, from images taken by different smartphones of the same model and camera software. When the system gets an image from the account in query, the system can derive its camera fingerprint $\hat{\mathbf{K}}_Q$ and compute the test statistic $\text{Cor}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_{Z_i})$. Since the fingerprints are from different smartphones, $\text{Cor}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_{Z_i})$ contains only the software noise component. The system can then compute the mean of $\text{Cor}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_{Z_i})$, $i = 1, \dots, n_Z$ as the estimation of software noise correlation.

Alleviating software noise correlation: There are two approaches to alleviate the software noise correlation: (i) Subtraction and (ii) Division. For subtraction, we can use the original test statistic to subtract the estimated software noise correlation. That is $\text{Cor}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) - \text{Cor}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_Z)$. For division, we can calculate a camera fingerprint to software noise ratio to determine whether two fingerprints match or not, $\text{Cor}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) / \text{Cor}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_Z)$. According to our analysis in Section V-F, division outperforms subtraction in stability, especially when images with various resolutions are captured by different device models.

C. Fingerprint to Software Noise Ratio—FITS

Given the existence of software noise, we propose new test statistics, fingerprint to software noise ratio (FITS) for fingerprint matching,

$$\text{FITS_Cor}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) = \frac{\text{Cor}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R)}{\text{Cor}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_Z)}. \quad (21)$$

That is, we need to first estimate the software noise correlation $\text{Cor}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_Z)$ and then derive the ratio of the polluted $\text{Cor}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R)$ over the estimated software noise correlation. We first show the new statistic based on the CC as follows,

$$\begin{aligned} \text{FITS_CC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) &= \frac{\text{CC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R)}{\text{CC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_Z)} \\ &= \frac{\alpha_Q \alpha_R \mathbf{K}_{Qm} \odot \mathbf{K}_{Rm} + \beta_Q \beta_R \mathbf{S}_{Qm} \odot \mathbf{S}_{Rm} + \varphi_{QR}}{\alpha_Q \alpha_Z \mathbf{K}_{Qm} \odot \mathbf{K}_{Zm} + \beta_Q \beta_Z \mathbf{S}_{Qm} \odot \mathbf{S}_{Zm} + \varphi_{QZ}}, \end{aligned} \quad (22)$$

where $\mathbf{K}_{Qm}$ and $\mathbf{K}_{Zm}$ are not matched because they are estimated from images captured by different devices. Because the mean of random noise φ_{QR} , φ_{QZ} and $\mathbf{K}_{Qm} \odot \mathbf{K}_{Zm}$ is statistically close to zero, we ignore them and substitute (22)

into (23),

$$\begin{aligned} \text{FITS_CC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) \\ \approx \frac{\alpha_Q \alpha_R \mathbf{K}_{Qm} \odot \mathbf{K}_{Rm} + \beta_Q \beta_R \mathbf{S}_{Qm} \odot \mathbf{S}_{Rm}}{\beta_Q \beta_Z \mathbf{S}_{Qm} \odot \mathbf{S}_{Zm}}. \end{aligned} \quad (23)$$

Because $\mathbf{K}_{Qm}$, $\mathbf{K}_{Rm}$, and $\mathbf{K}_{Zm}$ are estimated from images taken by different smartphones of the same model and camera software, $\mathbf{S}_{Qm}$, $\mathbf{S}_{Rm}$, and $\mathbf{S}_{Zm}$ behave identically. The value of $\beta_Q \beta_R \mathbf{S}_{Qm} \odot \mathbf{S}_{Rm}$ is statistically similar to $\beta_Q \beta_Z \mathbf{S}_{Qm} \odot \mathbf{S}_{Zm}$. FITS_CC can be expressed as follows:

$$\text{FITS_CC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) \approx \frac{\alpha_Q \alpha_R \mathbf{K}_{Qm} \odot \mathbf{K}_{Rm}}{\beta_Q \beta_Z \mathbf{S}_{Qm} \odot \mathbf{S}_{Zm}} + 1. \quad (24)$$

From FITS_CC in (24) for polluted images, we can observe: (i) For non-matching images, FITS_CC approaches 1 since the images are from different cameras and $\mathbf{K}_{Qm} \odot \mathbf{K}_{Rm}$ approaches zero. (ii) FITS_CC is bigger than 1 for matching images, since the images are taken from the same camera and $\mathbf{K}_{Qm} \odot \mathbf{K}_{Rm}$ exceeds zero.

Similarly, new statistic based on NCC is given as follows,

$$\begin{aligned} \text{FITS_NCC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) &= \frac{\text{NCC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R)}{\text{NCC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_Z)} \\ &= \frac{\|\hat{\mathbf{K}}_{Zm}\|}{\|\hat{\mathbf{K}}_{Rm}\|} \\ &\quad \cdot \frac{\alpha_Q \alpha_R \mathbf{K}_{Qm} \odot \mathbf{K}_{Rm} + \beta_Q \beta_R \mathbf{S}_{Qm} \odot \mathbf{S}_{Rm} + \varphi_{QR}}{\alpha_Q \alpha_Z \mathbf{K}_{Qm} \odot \mathbf{K}_{Zm} + \beta_Q \beta_Z \mathbf{S}_{Qm} \odot \mathbf{S}_{Zm} + \varphi_{QZ}}. \end{aligned} \quad (25)$$

After ignoring $\mathbf{K}_{Qm} \odot \mathbf{K}_{Zm}$ and other random noises φ_{QR} and φ_{QZ} , we then have:

$$\begin{aligned} \text{FITS_NCC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) &\approx \frac{\|\hat{\mathbf{K}}_{Zm}\| \alpha_Q \alpha_R \mathbf{K}_{Qm} \odot \mathbf{K}_{Rm}}{\|\hat{\mathbf{K}}_{Rm}\| \beta_Q \beta_Z \mathbf{S}_{Qm} \odot \mathbf{S}_{Zm}} \\ &\quad + \frac{\|\hat{\mathbf{K}}_{Zm}\| \beta_Q \beta_R \mathbf{S}_{Qm} \odot \mathbf{S}_{Rm}}{\|\hat{\mathbf{K}}_{Rm}\| \beta_Q \beta_Z \mathbf{S}_{Qm} \odot \mathbf{S}_{Zm}}. \end{aligned} \quad (26)$$

Then, we substitute (26) into (27) because of the same reason from (23) into (24):

$$\begin{aligned} \text{FITS_NCC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) &\approx r_{\text{FITS_NCC}} \\ &\quad \cdot \frac{\alpha_Q \alpha_R \mathbf{K}_{Qm} \odot \mathbf{K}_{Rm}}{\beta_Q \beta_Z \mathbf{S}_{Qm} \odot \mathbf{S}_{Zm}} + 1, \end{aligned} \quad (27)$$

where $r_{\text{FITS_NCC}} = \|\hat{\mathbf{K}}_{Zm}\| / \|\hat{\mathbf{K}}_{Rm}\|$.

The equation corresponding to PCE can be written as follows,

$$\begin{aligned} \text{FITS_PCE}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) &= \frac{\text{PCE}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R)}{\text{PCE}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_Z)} \\ &= \frac{\frac{\text{CC}(\mathbf{K}_Q, \mathbf{K}_R, \mathbf{s}_{\text{peak}})^2}{\frac{1}{mn-|N|} \sum_{\mathbf{s}, \mathbf{s} \notin N} \text{CC}(\mathbf{K}_Q, \mathbf{K}_R, \mathbf{s})^2}}{\frac{\text{CC}(\mathbf{K}_Q, \mathbf{K}_Z, \mathbf{s}_{\text{peak}})^2}{\frac{1}{mn-|N|} \sum_{\mathbf{s}, \mathbf{s} \notin N} \text{CC}(\mathbf{K}_Q, \mathbf{K}_Z, \mathbf{s})^2}} \end{aligned} \quad (28)$$

Because $\text{CC}(\mathbf{K}_Q, \mathbf{K}_R, \mathbf{s})$ and $\text{CC}(\mathbf{K}_Q, \mathbf{K}_Z, \mathbf{s})$ are both random noises with similar strength. Equation (28) can be simplified as:

$$\begin{aligned} \text{FITS_PCE}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) &\approx \frac{\text{CC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R, \mathbf{s}_{\text{peak}})^2}{\text{CC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_Z, \mathbf{s}_{\text{peak}})^2} \\ &= \frac{r_{\text{FITS_PCE_QR}}}{r_{\text{FITS_PCE_QZ}}} \\ &\quad \cdot \frac{(\alpha_Q \alpha_R \mathbf{K}_{Qm} \odot \mathbf{K}_{Rm} + \beta_Q \beta_R \mathbf{S}_{Qm} \odot \mathbf{S}_{Rm} + \varphi_{QR})_{\text{peak}}^2}{(\alpha_Q \alpha_Z \mathbf{K}_{Qm} \odot \mathbf{K}_{Zm} + \beta_Q \beta_Z \mathbf{S}_{Qm} \odot \mathbf{S}_{Zm} + \varphi_{QZ})_{\text{peak}}^2} \\ &\approx \frac{r_{\text{FITS_PCE_QR}}}{r_{\text{FITS_PCE_QZ}}} \\ &\quad \cdot \frac{(\alpha_Q \alpha_R \mathbf{K}_{Qm} \odot \mathbf{K}_{Rm} + \beta_Q \beta_R \mathbf{S}_{Qm} \odot \mathbf{S}_{Rm})_{\text{peak}}^2}{(\beta_Q \beta_Z \mathbf{S}_{Qm} \odot \mathbf{S}_{Zm})_{\text{peak}}^2}, \end{aligned} \quad (29)$$

where, $r_{\text{FITS_PCE_QR}} =$

$$\frac{\sum_{\mathbf{s}, \mathbf{s} \notin N} (\alpha_Q \alpha_R \mathbf{K}_{Qm} \odot \mathbf{K}_{Rm_s} + \beta_Q \beta_R \mathbf{S}_{Qm} \odot \mathbf{S}_{Rm_s} + \varphi_{QR_s})^2}{mn - |N|}, \quad (31)$$

$r_{\text{FITS_PCE_QZ}} =$

$$\frac{\sum_{\mathbf{s}, \mathbf{s} \notin N} (\alpha_Q \alpha_Z \mathbf{K}_{Qm} \odot \mathbf{K}_{Zm_s} + \beta_Q \beta_Z \mathbf{S}_{Qm} \odot \mathbf{S}_{Zm_s} + \varphi_{QZ_s})^2}{mn - |N|}. \quad (32)$$

$r_{\text{FITS_PCE_QZ}}$ and $r_{\text{FITS_PCE_QR}}$ are random noise because the shifted fingerprints $\mathbf{K}_{Rm_s}$ and $\mathbf{K}_{Zm_s}$ are not matched with camera fingerprint in query $\mathbf{K}_{Qm}$. Equation (25) and (30) show that if fingerprints $\mathbf{K}_{Qm}$ and $\mathbf{K}_{Rm}$ are captured by different cameras, $\mathbf{K}_{Qm} \odot \mathbf{K}_{Rm}$ approaches zero. Since the camera apps are the same, the result is merely the ratio of two test statistics of software noise and approaches 1. On the contrary, $\mathbf{K}_{Qm} \odot \mathbf{K}_{Rm}$ will be higher than zero and the result will exceed 1 for matched camera fingerprints.

Considering the sign of the PCE and the way of our software noise estimation, we can rewrite FITS_PCE as follows,

$$\begin{aligned} \text{FITS_PCE}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) &= \frac{\text{PCE}_{\text{sgn}}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R)}{\text{mean}_Z(\text{PCE}_{\text{sgn}}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_Z))} \\ &= \frac{\text{PCE}_{\text{sgn}}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R)}{\frac{1}{n_Z} \sum_{i=1}^{n_Z} \text{PCE}_{\text{sgn}}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_{Z_i})}. \end{aligned} \quad (33)$$

We can derive other new test statistics similarly as follows, which demonstrate similar properties to FITS_PCE while the computation complexity is lower as discussed in Section II-B.

$$\text{FITS_CC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) = \frac{\text{CC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R)}{\frac{1}{n_Z} \sum_{i=1}^{n_Z} \text{CC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_{Z_i})}, \quad (35)$$

$$\text{FITS_NCC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) = \frac{\text{NCC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R)}{\frac{1}{n_Z} \sum_{i=1}^{n_Z} \text{NCC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_{Z_i})}. \quad (36)$$

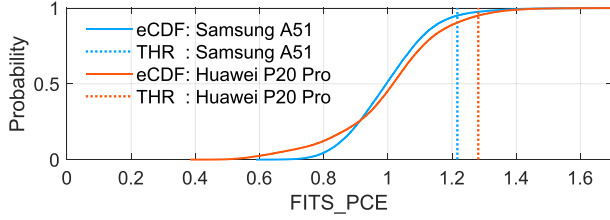


Fig. 7. eCDF of H0 distribution of FITS_PCE for Samsung A51 and Huawei P20 Pro.

D. Threshold Setting

Our test statistics such as FITS_PCE approach 1 when two images (and thus their fingerprints) are from different cameras. In practice, we need a threshold to reject the null hypothesis (i.e., the hypothesis two fingerprints are different) in the hypothesis testing problem in Formula (4). We can derive the H0 distribution of a test statistic and derive the threshold at a given false acceptance rate (FAR).

When two fingerprints are from different cameras, since our test statistics always approach 1, a common threshold can be used for any cameras for fingerprint matching. Fig. 7 shows the empirical cumulative distribution function (eCDF) curves of the test statistic FITS_PCE for Samsung A51 and Huawei P20 Pro smartphones. The legend THR refers to the threshold line with FAR=5%. We can see the two thresholds are very close to each other. A common threshold can then be derived. We have similar observations for test statistics FITS_CC and FITS_NCC.

E. Generalized Fingerprint to Software Noise Ratio—FITS⁺

When software noise is different between $\hat{\mathbf{K}}_Q$, $\hat{\mathbf{K}}_R$ or no software noise exists, FITS in (21) cannot be used for fingerprint matching since the software noise correlation component $\text{Cor}(\hat{\mathbf{S}}_Q, \hat{\mathbf{S}}_Z)$ approaches zero.

We want a test statistic that can be used for images with or without software noise. We propose a new test statistic FITS⁺ as follows,

$$\text{FITS}^+_{\text{Cor}}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) = \frac{\text{Cor}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) + C}{\text{Cor}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_Z) + C}, \quad (37)$$

where $\text{Cor}(\cdot)$ is the test statistic, i.e., CC in (5), NCC in (6) or PCE in (7), and C is a constant.

Take the PCE based test statistic as an example, FITS⁺_PCE is given as follows,

$$\text{FITS}^+_{\text{PCE}}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) = \frac{\text{PCE}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) + C}{\text{PCE}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_Z) + C}, \quad (38)$$

adding C to the denominator and numerator of FITS⁺_PCE is equivalent to introducing extra software noise into the image. Substitute (30) into (38),

$$\begin{aligned} \text{FITS}^+_{\text{PCE}}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) \\ \approx \frac{r_{\text{FITS_PCE_QR}}}{r_{\text{FITS_PCE_QZ}}} \end{aligned}$$

$$\cdot \frac{(\alpha_Q \alpha_R \mathbf{K}_{Qm} \odot \mathbf{K}_{Rm} + \beta_Q \beta_R \mathbf{S}_{Qm} \odot \mathbf{S}_{Rm})^2_{\text{peak}} + C}{(\beta_Q \beta_Z \mathbf{S}_{Qm} \odot \mathbf{S}_{Zm})^2_{\text{peak}} + C}, \quad (39)$$

we make the following observations from (39): (i) For images without the same software noise, $\beta_Q \beta_R \mathbf{S}_{Qm} \odot \mathbf{S}_{Rm}$ and $\beta_Q \beta_Z \mathbf{S}_{Qm} \odot \mathbf{S}_{Zm}$ both approach zero. If the two images are from different cameras, $\mathbf{K}_{Qm} \odot \mathbf{K}_{Rm}$ approaches zero and FITS⁺_PCE approaches 1. If the two images are from the same camera, $\mathbf{K}_{Qm} \odot \mathbf{K}_{Rm}$ is large and FITS⁺_PCE is bigger than 1; (ii) In the case of images with the same software noise, FITS⁺_PCE behaves similarly. FITS⁺_PCE approaches 1 if the two images are from different cameras. FITS⁺_PCE is bigger than 1 if the two images are from the same camera. Therefore, the new test statistic FITS⁺_PCE can handle both cases with(out) software noise through appropriate C .

Considering the estimation of software noise in (34), we rewrite (38) as follows,

$$\text{FITS}^+_{\text{PCE}}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) = \frac{\text{PCE}_{\text{sgn}}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) + C}{\frac{1}{n_Z} \sum_{i=1}^{n_Z} \text{PCE}_{\text{sgn}}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_{Z_i}) + C}. \quad (40)$$

Selecting C: Apparently, C in FITS⁺_PCE cannot be too large. Otherwise, FITS⁺_PCE always approaches 1. Since C plays a role of extra noise, it shall be far smaller than $\text{PCE}(\mathbf{K}_Q, \mathbf{K}_R)$ given the two images are from the same camera. We can determine C empirically to achieve the optimal fingerprint matching.

We can derive generalized versions of other test statistics similarly as follows:

$$\text{FITS}^+_{\text{CC}}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) = \frac{\text{CC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) + C}{\frac{1}{n_Z} \sum_{i=1}^{n_Z} \text{CC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_{Z_i}) + C}, \quad (41)$$

$$\text{FITS}^+_{\text{NCC}}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) = \frac{\text{NCC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) + C}{\frac{1}{n_Z} \sum_{i=1}^{n_Z} \text{NCC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_{Z_i}) + C}. \quad (42)$$

F. Discussion

In this section, we analyze the way to alleviate software noise correlation through subtraction and compare it with the way through division.

1) *Using Subtraction to Calculate FITS_CC:* Using subtraction to calculate FITS_CC can be expressed as follows,

$$\begin{aligned} \text{FITS_CC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) \\ = \text{CC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) - \text{CC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_Z) \\ = \alpha_Q \alpha_R \mathbf{K}_{Qm} \odot \mathbf{K}_{Rm} + \beta_Q \beta_R \mathbf{S}_{Qm} \odot \mathbf{S}_{Rm} \\ - \beta_Q \beta_Z \mathbf{S}_{Qm} \odot \mathbf{S}_{Zm} + \varphi_{QRZ}, \end{aligned} \quad (43)$$

where

$$\varphi_{QRZ} = \varphi_{QR} + \varphi_{QZ} - \alpha_Q \alpha_Z \mathbf{K}_{Qm} \odot \mathbf{K}_{Zm}. \quad (44)$$

Please note that $\mathbf{K}_{Qm}$ and $\mathbf{K}_{Zm}$ are hardware fingerprints from different devices. Therefore $\mathbf{K}_{Qm} \odot \mathbf{K}_{Zm}$ is included in the combination of random noises φ_{QRZ} . Since the mean of random noise φ_{QRZ} is statistically close to zero, we ignore φ_{QRZ} and (43) can be further simplified as:

$$\begin{aligned} \text{FITS_CC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) &\approx \alpha_Q \alpha_R \mathbf{K}_{Qm} \odot \mathbf{K}_{Rm} \\ &\quad + \beta_Q \beta_R \mathbf{S}_{Qm} \odot \mathbf{S}_{Rm} \\ &\quad - \beta_Q \beta_Z \mathbf{S}_{Qm} \odot \mathbf{S}_{Zm}. \end{aligned} \quad (45)$$

The strength of hardware fingerprint $\mathbf{K}$ and software noise $\mathbf{S}$ is sensitive to picture resolution. If we resize the input pictures, the result of (45) is changed accordingly. Since the numerator and denominator are synchronously changed in (23), the result remains stable.

2) *Using Subtraction to Calculate FITS_NCC*: Using subtraction to calculate FITS_NCC is given as follows,

$$\begin{aligned} \text{FITS_NCC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) &= \text{NCC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) - \text{NCC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_Z) \\ &= \frac{\alpha_Q \alpha_R \mathbf{K}_{Qm} \odot \mathbf{K}_{Rm} + \beta_Q \beta_R \mathbf{S}_{Qm} \odot \mathbf{S}_{Rm} + \varphi_{QR}}{\|\hat{\mathbf{K}}_{Qm}\| \|\hat{\mathbf{K}}_{Rm}\|} \\ &\quad - \frac{\alpha_Q \alpha_Z \mathbf{K}_{Qm} \odot \mathbf{K}_{Zm} + \beta_Q \beta_Z \mathbf{S}_{Qm} \odot \mathbf{S}_{Zm} + \varphi_{QZ}}{\|\hat{\mathbf{K}}_{Qm}\| \|\hat{\mathbf{K}}_{Zm}\|}. \end{aligned} \quad (46)$$

We ignore zero-mean random noises φ_{QR} , $\mathbf{K}_{Qm} \odot \mathbf{K}_{Zm}$, and φ_{QZ} , and have:

$$\begin{aligned} \text{FITS_NCC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) &\approx \frac{\alpha_Q \alpha_R \mathbf{K}_{Qm} \odot \mathbf{K}_{Rm} + \beta_Q \beta_R \mathbf{S}_{Qm} \odot \mathbf{S}_{Rm}}{\|\hat{\mathbf{K}}_{Qm}\| \|\hat{\mathbf{K}}_{Rm}\|} \\ &\quad - \frac{\beta_Q \beta_Z \mathbf{S}_{Qm} \odot \mathbf{S}_{Zm}}{\|\hat{\mathbf{K}}_{Qm}\| \|\hat{\mathbf{K}}_{Zm}\|}. \end{aligned} \quad (47)$$

In (47), the result is relevant to $\|\hat{\mathbf{K}}_{Qm}\|$, the noise residual strength of the picture, which may result in instability. Instead, (26) doesn't contain $\|\hat{\mathbf{K}}_{Qm}\|$, and it isn't sensitive to the input picture's hardware fingerprint strength.

3) *Using Subtraction to Calculate FITS_PCE*: The equation of subtraction is given as follows, $\text{FITS_PCE}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) = \text{PCE}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) - \text{PCE}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_Z)$

$$\begin{aligned} \text{FITS_PCE}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) &= \text{PCE}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) - \text{PCE}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_Z) \\ &= \frac{\text{CC}(\mathbf{K}_Q, \mathbf{K}_R, \mathbf{s}_{\text{peak}})^2}{\frac{1}{mn-|N|} \sum_{\mathbf{s}, \mathbf{s} \notin N} \text{CC}(\mathbf{K}_Q, \mathbf{K}_R, \mathbf{s})^2} \\ &\quad - \frac{\text{CC}(\mathbf{K}_Q, \mathbf{K}_Z, \mathbf{s}_{\text{peak}})^2}{\frac{1}{mn-|N|} \sum_{\mathbf{s}, \mathbf{s} \notin N} \text{CC}(\mathbf{K}_Q, \mathbf{K}_Z, \mathbf{s})^2}, \end{aligned} \quad (48)$$

where $\text{CC}(\mathbf{K}_Q, \mathbf{K}_R, \mathbf{s})$ and $\text{CC}(\mathbf{K}_Q, \mathbf{K}_Z, \mathbf{s})$ are both random noises with similar strength, because the shifted fingerprints $\mathbf{K}_R$ and $\mathbf{K}_Z$ are not correlated with the camera fingerprint $\mathbf{K}_Q$, and these three fingerprints have the same image resolution.

Equation (48) can be simplified as follows,

$$\begin{aligned} \text{FITS_PCE}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) &\approx \frac{\text{CC}(\mathbf{K}_Q, \mathbf{K}_R, \mathbf{s}_{\text{peak}})^2 - \text{CC}(\mathbf{K}_Q, \mathbf{K}_Z, \mathbf{s}_{\text{peak}})^2}{\frac{1}{mn-|N|} \sum_{\mathbf{s}, \mathbf{s} \notin N} \text{CC}(\mathbf{K}_Q, \mathbf{K}_R, \mathbf{s})^2}. \end{aligned} \quad (49)$$

In (49), the denominator may be influenced by the model of devices, which makes setting a certain threshold for multiple models a difficult task. There is no such problem in (29).

4) *Summary*: For CC, NCC and PCE algorithms, using subtraction to calculate FITS may result in fluctuation of results, which may degrade the performance of our fingerprint matching algorithm. Instead, using division facilitates threshold choosing and enhances the stability.

VI. EVALUATION

In this section, we conduct a series of experiments to verify the abilities of our new test statistics and the performance of our account linking system. We first introduce the experiment setup, metrics and terms used in the evaluation. Next we demonstrate the effectiveness of our new fingerprint matching approaches. We then show the efficiency of our fingerprint matching techniques and their robustness by artificially introducing software noise into images through image processing techniques including digital watermarking, sticker, and camera fingerprint concealment. At last, we show the performance of our approaches in user linking, and compare our approaches and conventional fingerprint matching approaches.

A. Experiment Setup, Metrics and Terms

Camera devices: A large number of smartphones of various models and digital cameras are used in our experiments, including thirty Samsung A51, thirty Huawei P20 Pro, and thirty VIVO IQOO U1X smartphones. For better demonstration of a particular phenomenon, each experiment may use a different set of devices and we will introduce the corresponding experiment setup later.

Images: We capture images such as buildings, sky, and cars in various environments. These images are taken at different ambient temperatures and humidity from 2021 to 2022. All devices were held in the same orientation, with a forward-facing grip. In the settings menu, all manually adjustable optimization items were turned off, such as HDR and AI enhancements.

Computing device: We process experiment data and run fingerprint matching algorithms in Matlab on a desktop with an 8-core CPU of 2.9 GHz.

Metrics: We use the area under the ROC curve (AUC), equal error rate (EER), and the EER threshold to measure effectiveness and robustness of fingerprint matching approaches. AUC has a maximum value of 1. The larger the AUC, the better the fingerprint matching. A large AUC implies that a fingerprint matching approach can achieve a large true positive rate at a low false positive rate. The smaller the EER, the better the fingerprint matching. At the EER threshold, a fingerprint matching approach performs best considering both false positive and false negative rates.

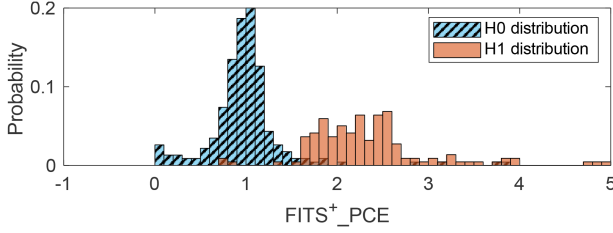


Fig. 8. H0 and H1 distributions of $\text{FITS}^+_{\text{PCE}}$. H0 distribution's mean = 1.

Other Terms: A camera fingerprint can be extracted from one or multiple images via the Maximum Likelihood Estimator (MLE). We denote the number of images used to produce a fingerprint as n_{mle} as shown in (3). For FITS and FITS^+ , we denote the number of fingerprints for estimating software noise as n_z .

B. Effectiveness of Test Statistics

We now show FITS and FITS^+ are much superior to the conventional fingerprint matching approaches, which are introduced in Section II, in the presence of software noise.

1) **EER:** In order to show the better performance of our approaches in the complex software noise situation, we compare the performance of the PCE based test statistics, $\text{FITS}^+_{\text{PCE}}$ in (40) and PCE in (8). The experiment setup is the same as the setup in Section V-A, including images from different smartphones and digital cameras. Some images are influenced by software noise but others are not.

Fig. 8 shows the H0 and H1 distributions of $\text{FITS}^+_{\text{PCE}}$, while Fig. 6 in Section V-A shows the H0 and H1 distributions of PCE. The EER by $\text{FITS}^+_{\text{PCE}}$ is 4.78% compared with 45.21% by PCE, which indicates our approach is more effective in the scenario where software noise may or may not exist.

2) **AUC:** We now demonstrate FITS and FITS^+ perform much better than conventional test statistics in terms of AUC. Without loss of generality, we use 30 Samsung A51 and 30 Huawei P20 Pro in this set of experiments.

Fig. 9 shows the AUC of Samsung A51 versus n_{mle} (i.e., number of images to estimate a camera fingerprint) and n_z (i.e., the number of images to estimate the software noise). When $n_{mle} = 20$ and $n_z = 10$, FITS_CC achieves a perfect AUC of 1.000, while CC achieves a much lower AUC of 0.840.

Fig. 9(a) shows that the AUC increases with increasing n_{mle} . When n_{mle} increases from 1 to 20, the AUC of CC and FITS_CC improves by 13.97% and 25.64% when $n_z = 10$. It can also be observed that the higher the n_z , the higher the AUC. For instance, the AUC of FITS_CC at $n_{mle} = 5$ is 0.836, 0.908, and 0.926 when $n_z = 1, 3$ and 10 respectively.

Compared with Samsung A51, Huawei P20 Pro has a H1 distribution with a smaller mean and thus has a weaker camera fingerprint in our experiments. Weak fingerprints produce smaller AUC. However, Fig. 10 shows that FITS and FITS^+ still perform much better than conventional approaches. The AUC of FITS_CC , FITS_NCC and FITS_PCE is 15.06%, 21.45%, and 21.36% higher than the AUC of conventional methods at

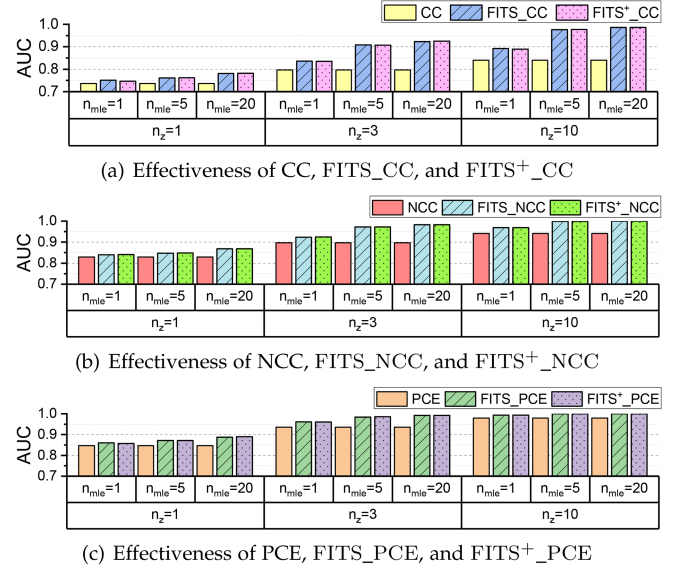


Fig. 9. AUC of Samsung A51. FITS and FITS^+ outperform conventional approaches.

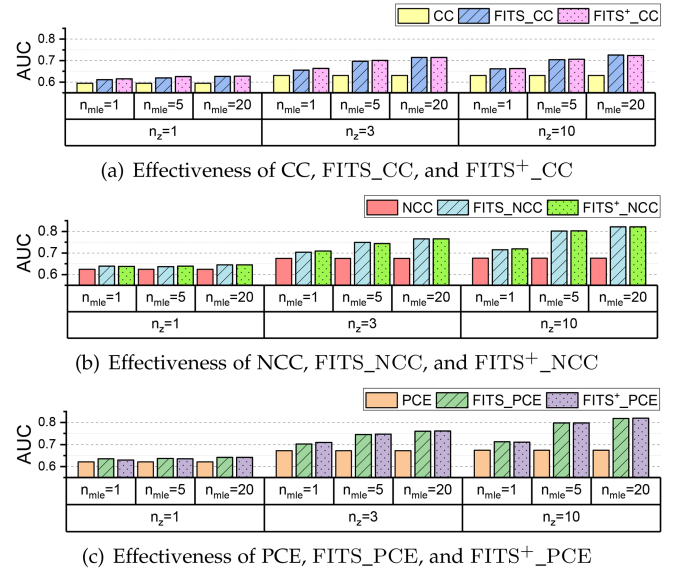


Fig. 10. AUC of Huawei P20 Pro. FITS and FITS^+ outperform conventional approaches.

$n_{mle} = 20$ and $n_z = 10$. The AUC produced by our approaches increases more quickly with increasing n_{mle} compared with conventional approaches. At $n_z = 10$, the AUC of NCC is 0.624 at $n_{mle} = 1$, 0.675 at $n_{mle} = 5$, and 0.676 at $n_{mle} = 20$, while the AUC of FITS_NCC is 0.645 at $n_{mle} = 1$, 0.765 at $n_{mle} = 5$, and 0.821 at $n_{mle} = 20$. When n_{mle} increases from 1 to 20, the AUC of NCC improves by 8.33% while the AUC of our FITS_NCC improves by 27.29%. The AUC also increases with n_z .

In terms of the AUC improvement, we have similar observations for FITS^+ on both Samsung A51 and Huawei P20 Pro. According to our observation, FITS performs a bit better than

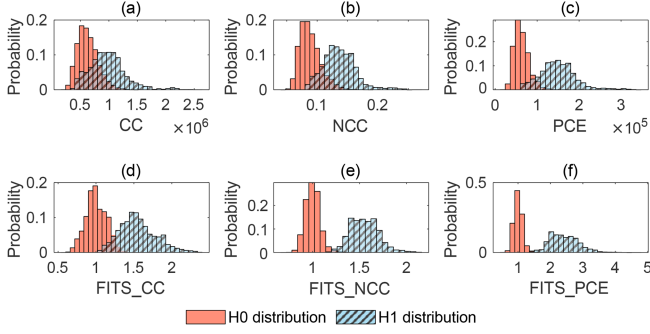


Fig. 11. H0 and H1 Distributions of different test statistics. Our FITS test statistics produce less overlapping H0 and H1 distributions and thus better AUC.

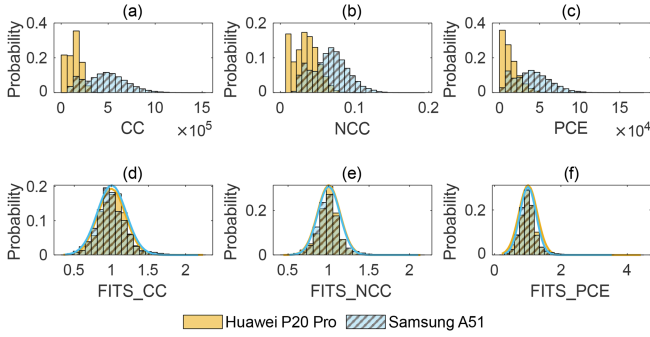


Fig. 12. H0 distributions of test statistics for Samsung A51 and Huawei P20 Pro. Our FITS test statistics produce H0 distributions with mean around 1.

FITS⁺. We recommend one shall consider the FITS approach if the existence of software noise is known.

We now show why FITS performs better in terms of the AUC than the conventional approaches. Fig. 11 shows the H0 and H1 distributions of the six approaches for Samsung A51 at $n_{mle} = 20$ and $n_Z = 10$. It can be observed that the H0 and H1 distributions overlap very much for each conventional approach. However, for our FITS approach, the H0 and H1 distributions do not overlap much as shown in Fig. 11(d). The separation between the H0 and H1 distribution results in better performance for FITS approaches.

3) *H0 Distribution of FITS With Mean Around 1*: In Section V, we show that FITS approaches 1 when two images are from different cameras. Our experiments show the correctness of this theory: The H0 distribution of a FITS test statistic has a mean around 1 in spite of different device models, n_{mle} and n_Z . In our experiments, the H0 distributions also demonstrate smaller variance with increasing n_{mle} and n_Z .

Fig. 12 shows that the H0 distribution of our test statistics (FITS_CC, FITS_NCC, FITS_PCE) is not sensitive to the device model. The H0 distributions for Samsung A51 and Huawei P20 Pro both have a mean around 1 and overlap with each other. On the contrary, the H0 distributions of a conventional test statistic (CC, NCC, or PCE) for Samsung A51 and Huawei P20 Pro are separate and have different means.

Fig. 13 shows the H0 distribution of our test statistics (FITS_CC, FITS_NCC, FITS_PCE) is not sensitive to n_{mle} .

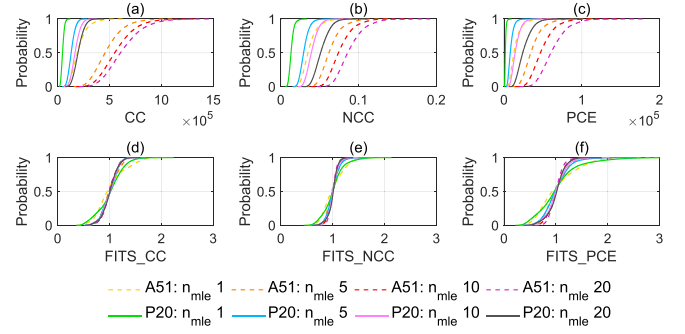


Fig. 13. CDF curves of H0 distribution with different n_{mle} when $n_Z = 10$. Our test statistics have a mean around 1 in spite of different n_{mle} .

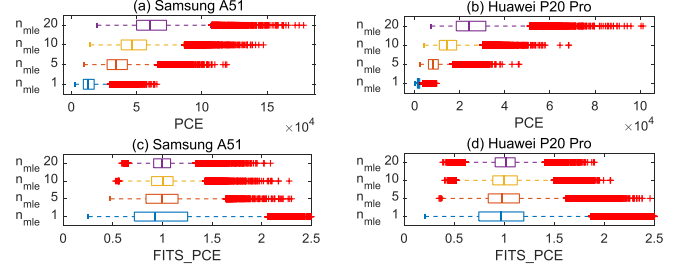


Fig. 14. Box plots of PCE and FITS_PCE of Samsung A51 and Huawei P20 Pro with $n_Z = 10$. FITS_PCE shows a smaller interquartile range with increasing n_{mle} .

This figure shows the eCDF curves of H0 distributions at different n_{mle} when $n_Z = 10$, where A51 and P20 refer to Samsung A51 and Huawei P20 Pro. The eCDF curves of conventional test statistics vary much when n_{mle} increases from 1 to 20. On the contrary, the eCDF curves of our test statistics overlap and have a mean around 1 for different n_{mle} . That is, the eCDFs show a probability of 0.5 when the test statistic on the horizontal axis is around 1.

Fig. 14 shows the H0 distribution has a smaller variance with larger n_{mle} . For brevity and clarity, we use box plots to represent the distribution of PCE and FITS_PCE. The interquartile range (implying variance) of the conventional PCE increases with increasing n_{mle} . By contrast, the interquartile range of FITS_PCE decreases with increasing n_{mle} . This will benefit the selection of a uniform threshold.

Fig. 15 shows that the H0 distributions of our FITS test statistics have a mean around 1 at different n_Z for different device models. Fig. 16 demonstrates that the interquartile range of the H0 distribution decreases with increasing n_Z .

C. Efficiency

Fig. 17 compares the efficiency of fingerprint extraction and matching with three conventional and three FITS approaches on Huawei P20 Pro and VIVO IQOO U1X. For each test statistic, we perform fingerprint extraction and matching on 100 pairs of images. The red dash line in Fig. 17 indicates the average time spent on camera fingerprint extraction. The time needed for extraction is 4.845 and 6.771 seconds on average for images

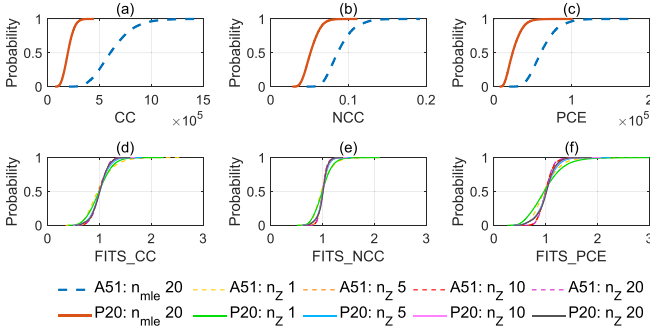


Fig. 15. CDF curves of H0 distributions with different n_Z and $n_{mle} = 20$. Our FITS test statistics have a mean around 1 in spite of different n_Z .

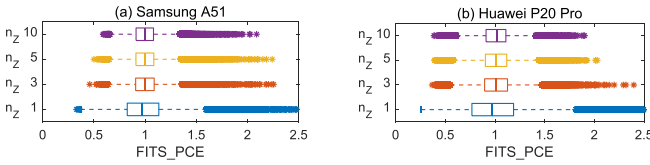


Fig. 16. Box plots of FITS_PCE of Samsung A51 and Huawei P20 Pro at $n_{mle} = 20$. All H0 distributions have a mean around 1 as n_Z increases.

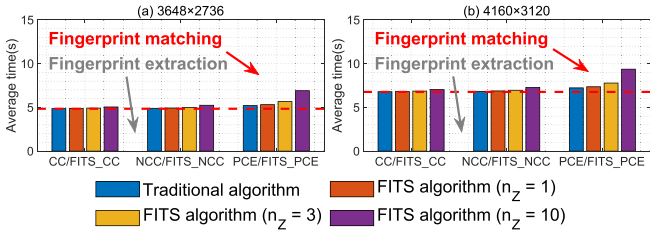


Fig. 17. Efficiency of conventional and FITS approaches with different n_Z . The area below the red dash line indicates the fingerprint extraction time.

with the resolution of 3648×2736 and 4160×3120 respectively. The combined time of extraction and matching with the three conventional approaches is 4.863, 4.896, 5.213 seconds for images of 3648×2736 , and 6.796, 6.837, 7.234 seconds for images of 4160×3120 . It can be observed that fingerprint matching is very efficient compared with fingerprint extraction. The fingerprint extraction and matching time increases a little with increasing n_Z for FITS_PCE, while the time increase for FITS_CC and FITS_NCC is not obvious.

Since fingerprint extraction is computationally intensive, we can use GPU to speed up this process. To this end, we convert the image matrix format from NumPy to CuPy and parallelize the fingerprint extraction algorithm for GPU. We conduct experiments on a server with Tesla A100, running CUDA Version 11.2. GPU for fingerprint extraction can achieve an average time of 0.478 seconds and 0.597 seconds for images of 3648×2736 and 4160×3120 , 10 times faster than the CPU-based method implemented in Matlab.

D. Robustness

We want to investigate how conventional and our fingerprint matching approaches perform under various malicious evasions such as image processing practices. The suspects may evade

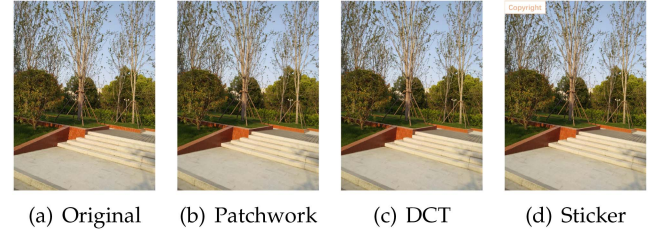


Fig. 18. Example original image and processed images with watermarking (Patchwork and DCT) and Sticker techniques.

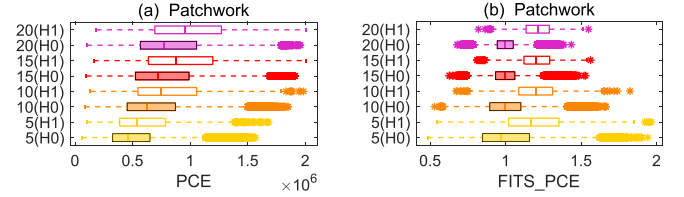


Fig. 19. Box plots of PCE under Patchwork with $n_Z = 10$. The vertical axis label is formatted as $n_{mle}(H0/H1)$. The H0 distributions of FITS_PCE have a mean around 1 under different n_{mle} .

identity linking with post-processing techniques such as digital watermarking and sticker. The camera fingerprint concealment technique [32] is proposed to hide camera fingerprints in images and may be maliciously used by suspects to evade identity linking. These image processing techniques introduce software noise into images.

Thirty VIVO IQOO U1X smartphones are used in this set of experiments to take images, and their default camera app does not introduce software noise.

1) *Watermark*: Digital watermarking adds an invisible pattern into a photo. Patchwork [33] and DCT [31] are two classical watermarking algorithms used in this evaluation. The Sticker technique adds a visible sticker (icon/picture) into an image. Fig. 18 shows the example of an original image and its corresponding processed images with the above three techniques.

Table I shows the results of fingerprint matching on the processed images. Our experiments show that Patchwork produces stronger software noise than DCT and Sticker, which may heavily influence the performance of conventional matching algorithms. We first compare our approaches with conventional approaches under Patchwork. Table I shows that the AUC of CC, NCC, and PCE is very low at $n_{mle} = 20$. In contrast, FITS_CC, FITS_NCC, and FITS_PCE achieve higher AUC than conventional approaches. For instance, when $n_Z = 10$, the AUC of FITS_CC, FITS_NCC, and FITS_PCE is 28.54%, 42.76% and 46.65% higher than the AUC of CC, NCC, and PCE respectively. The AUC of our statistics increases rapidly with increasing n_Z . FITS algorithms need to estimate the software noise correlation, which is a process with randomness. The larger n_Z , the more accurate the estimation becomes. Therefore, when $n_Z = 1$, the AUC results for FITS_NCC and FITS_PCE do not reach 1.000 with Sticker as the post-solving algorithm. However, at higher n_Z values, the AUC can reach 1.000.

Fig. 19(a) illustrates the box plots of PCE under Patchwork. It can be observed that the H0 and H1 distributions overlap heavily

TABLE I
COMPARISON OF AUC BETWEEN CONVENTIONAL APPROACHES AND FITS ($n_{mle} = 20$) UNDER THE INFLUENCE OF WATERMARKING, STICKER, FINGERPRINT
CONCEALMENT TECHNIQUES

Post-solving algorithm	n_Z	CC	FITS_CC	NCC	FITS_NCC	PCE	FITS_PCE
Patchwork	1		0.677		0.639		0.643
	3		0.818		0.869		0.906
	5	0.655	0.829	0.622	0.882	0.628	0.915
	10		0.842		0.888		0.921
DCT	1, 3, 5, 10	1.000	1.000	1.000	1.000	1.000	1.000
Sticker	1	1.000	1.000	1.000	0.999	1.000	0.999
	3, 5, 10	1.000	1.000	1.000	1.000	1.000	1.000
CFP	1		0.636		0.644		0.644
	3		0.829		0.892		0.904
	5	0.628	0.840	0.635	0.904	0.638	0.917
	10		0.851		0.912		0.925

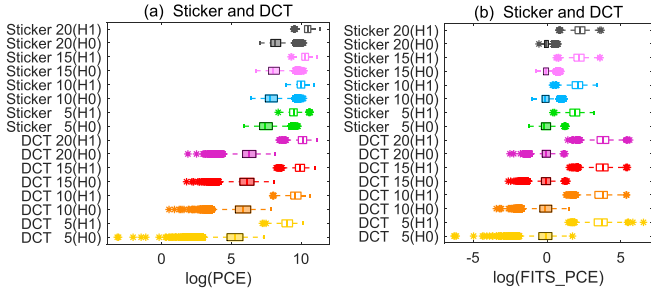


Fig. 20. Box plots of $\log(PCE)$ and $\log(FITS_PCE)$ under Sticker and DCT at different n_{mle} and $n_Z = 10$. The vertical axis label is formatted as Sticker/DCT n_{mle} (H0/H1). The H0 distributions of $FITS_PCE$ have a mean around 1.

under different n_{mle} . The interquartile range increases with increasing n_{mle} . In contrast, Fig. 19(b) shows that for $FITS_PCE$, H0 distributions have a mean around 1. The interquartile range decreases with increasing n_{mle} .

Fig. 20(a) gives the box plots of $\log(PCE)$ and Fig. 20(b) gives the box plots of $\log(FITS_PCE)$ under Sticker and DCT at different n_{mle} when $n_Z = 10$. Although both conventional approaches and our approaches can achieve an AUC of 1 under DCT and Sticker, the H0 distributions of our test statistic have a mean around 1. This allows us to use a uniform threshold for fingerprint matching. A uniform threshold for fingerprint matching is one salient advantage of our approaches.

2) *Camera Fingerprint Concealment*: The camera fingerprint concealment technique, called CFP, in [32] is proposed to defeat camera fingerprint matching for privacy protection. It adds artificial noise into all protected images, which could be maliciously used by suspects to hide their true identities.

Table I shows that the AUC of conventional test statistics is pretty low, which indicates that conventional algorithms are susceptible to CFP. For instance, the AUC of CC, NCC and PCE is 0.628, 0.635, and 0.638 at $n_{mle} = 20$. In comparison, we can see a significant improvement of AUC with our FITS test statistics. For example, the AUC of $FITS_PCE$ is improved by 44.98% to 0.925 compared with the conventional PCE at $n_Z = 10$.

We now use H0 and H1 distributions to intuitively show our approaches perform better under the influence of the camera fingerprint concealment technique. Fig. 21(a) shows large overlapping between the H0 and H1 distributions of the conventional

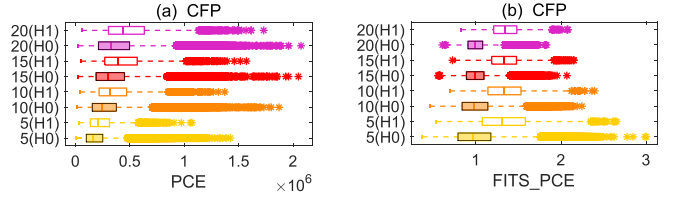


Fig. 21. Box plots of PCE and $FITS_PCE$ on VIVO IQOO U1X images processed by the fingerprint concealment technique—CFP. H0 and H1 distributions of PCE overlap a lot while H0 and H1 distributions of $FITS_PCE$ are separate.

PCE at different n_{mle} , and the interquartile range of H0 and H1 distributions increases with increasing n_{mle} . In contrast, as shown in Fig. 21(b), the H0 and H1 distributions of $FITS_PCE$ have much less overlapping, and the interquartile range decreases with increasing n_{mle} .

Please note that CFP always needs to inject a certain strength of fixed obfuscating noise, which is equivalent to software noise, into the images to achieve anonymization. FITS can eliminate the impact of such noise. Therefore, CFP cannot be easily adapted to evade FITS and achieve camera fingerprint concealment.

E. Fingerprint Matching Application—Account Linking

We now show the performance of FITS and $FITS^+$ approaches for account linking. We have introduced the typical workflow of an account linking system in Section III. For a new account linking system with FITS and $FITS^+$, the fingerprint extraction and fingerprint matching remain unchanged. We merely add an additional step, namely software noise determination, before the test statistic selection.

Software noise determination: We assume the system has a database of images and make the following assumptions: (i) The images are captured by cameras of potential device models that a user might use; (ii) For each device model, we collect many images. With these images, the system can determine whether a device model generates software noise, as observed in Section IV-A. According to Counterpoint [34], the shipments of smartphones is 2.488 billion units from Q3 2021 to Q2 2023, of which 1.726 billion units are from the top-five smartphone brand (Samsung, Apple, Xiaomi, OPPO, and VIVO). Therefore, the diversity of devices is limited. We think a system owner can obtain the necessary images and create an image database.

TABLE II
COMPARISON OF AUC, EER, AND THRESHOLDS OF USER LINKING BETWEEN CONVENTIONAL APPROACHES AND FITS(FITS⁺)

Account reference	Algorithm	AUC	Impro	EER(%)	Decre	AUC	Impro	EER(%)	Decre	Threshold@FAR=5%			
			Huawei P20 Pro				Samsung A51			Huawei P20 Pro	Samsung A51	Difference ratio	
6	CC	Conventional	0.932	-	13.66	-	0.967	-	9.16	-	8697.005	38471.007	4.423
		FITS	0.963	3.33%	9.17	32.87%	0.999	3.31%	1.48	83.88%	1.306	1.397	1.070
		FITS+	0.962	3.22%	9.17	32.87%	0.999	3.31%	1.83	79.99%	1.148	1.223	1.065
	NCC	Conventional	0.936	-	12.83	-	0.994	-	3.88	-	0.007	0.031	4.459
		FITS	0.963	2.88%	8.63	32.74%	1.000	0.60%	0.17	95.69%	1.297	1.329	1.025
		FITS+	0.963	2.88%	8.67	32.46%	1.000	0.60%	0.17	95.69%	1.316	1.179	0.896
	PCE	Conventional	0.937	-	12.83	-	0.994	-	3.92	-	142.317	2831.037	19.893
		FITS	0.963	2.77%	8.56	33.32%	1.000	0.60%	0.17	95.74%	1.603	1.696	1.058
		FITS+	0.963	2.77%	8.50	33.76%	1.000	0.60%	0.17	95.74%	1.686	1.365	0.810
8	CC	Conventional	0.949	-	12.33	-	0.980	-	7.00	-	10452.645	47071.309	4.503
		FITS	0.978	3.06%	7.33	40.54%	1.000	2.04%	1.33	80.96%	1.292	1.402	1.085
		FITS+	0.977	2.95%	7.99	35.22%	0.999	1.94%	1.67	76.19%	1.147	1.244	1.085
	NCC	Conventional	0.955	-	12.00	-	0.999	-	2.33	-	0.009	0.040	4.390
		FITS	0.979	2.51%	6.83	43.06%	1.000	0.10%	0.00	100.00%	1.293	1.431	1.107
		FITS+	0.979	2.51%	6.94	42.14%	1.000	0.10%	0.00	100.00%	1.294	1.159	0.896
	PCE	Conventional	0.955	-	11.67	-	0.999	-	1.95	-	233.659	4470.761	19.134
		FITS	0.979	2.51%	6.67	42.86%	1.000	0.10%	0.00	100.00%	1.611	2.008	1.247
		FITS+	0.979	2.51%	6.82	41.53%	1.000	0.10%	0.00	100.00%	1.644	1.314	0.799
10	CC	Conventional	0.961	-	11.50	-	0.981	-	7.33	-	11857.372	52479.469	4.426
		FITS	0.988	2.81%	5.17	55.07%	1.000	1.94%	1.33	81.82%	1.312	1.377	1.050
		FITS+	0.987	2.71%	5.50	52.17%	0.999	1.83%	1.33	81.82%	1.150	1.227	1.067
	NCC	Conventional	0.970	-	10.10	-	1.000	-	1.00	-	0.011	0.048	4.355
		FITS	0.990	2.06%	4.17	58.73%	1.000	0.00%	0.00	100.00%	1.303	1.419	1.089
		FITS+	0.990	2.06%	4.17	58.73%	1.000	0.00%	0.00	100.00%	1.267	1.141	0.901
	PCE	Conventional	0.970	-	9.86	-	1.000	-	0.83	-	350.731	6522.195	18.596
		FITS	0.991	2.16%	4.17	57.75%	1.000	0.00%	0.00	100.00%	1.627	2.052	1.261
		FITS+	0.991	2.16%	4.17	57.75%	1.000	0.00%	0.00	100.00%	1.575	1.272	0.808

Account reference n_{mle} =6, 8, and 10. The Impro and Decre columns indicate the increased or decreased percentage for FITS(FITS⁺) compared with conventional approaches. The difference ratio is the ratio between the thresholds of two device models for the same algorithm.

Test statistic selection: The system selects a proper test statistic according to the existence of software noise. With the help of the software noise dataset, the system can determine whether two camera fingerprints are polluted by the same software noise. When two fingerprints are polluted by the same software noise, the correlation between software noise is larger than zero, which affects fingerprint matching through conventional test statistics. Thus, if two fingerprints are polluted by the same software noise, the system can select FITS as the test statistic. When two fingerprints are not polluted by software noise or polluted by different software noise, the correlation between software noise tends to be zero. Therefore, the system can select conventional test statistics (CC, NCC, and PCE) or FITS⁺.

We use thirty Samsung A51 and Huawei P20 Pro to examine the stability of thresholds in this experiment. In comparison to Samsung A51, the camera fingerprint of Huawei P20 Pro is relatively weak. Since Facebook is the most used OSN in the world [35], we choose it as the OSN for the following experiments. We upload and download images which are originally captured by the default camera apps of both device models.

1) **AUC and EER:** Table II shows the results of user linking. We denote the camera fingerprint of an account as account reference fingerprint, and select $n_{mle} = 6, 8, 10$. We find in practice 6-10 images are good in this scenario, and similar observations are made in CFP, which uses around 5 images from each social account [32]. Since it is easy for conventional and our approaches to match two fingerprints not under the interference of software noise, we match account reference fingerprints from the same device model to compare the performance of conventional approaches, FITS, and FITS⁺. The experiment result shows that FITS and FITS⁺ perform better than conventional statistics (CC, NCC, and PCE) for both device models in AUC and EER. The *Impro* column and *Decre* column indicate the increased percentage and decreased percentage of FITS(FITS⁺) compared with conventional approaches. All AUCs of FITS and

FITS⁺ are increased compared with all conventional statistics for both Huawei P20 Pro and Samsung A51. Meanwhile, all EERs of FITS and FITS⁺ are decreased compared with three conventional statistics for Huawei P20 Pro and Samsung A51. The conventional statistics perform well when handling images already uploaded to Facebook. One possible reason is that part of the software noise has been mitigated in the OSN by certain processing. FITS and FITS⁺ are robust enough because they still perform better than conventional test statistics for AUCs and EERs in this situation.

2) **Stability of Threshold:** We calculate the thresholds at FAR=5% based on H0 distributions of two device models and various test statistics for comparison. The last column of Table II shows the stability of thresholds. The *difference ratio* column indicates the ratio between the thresholds of different device models for the same test statistic. For the same conventional test statistic, the thresholds vary dramatically between device models. For the same device model, the thresholds among the three conventional test statistics are extremely different. The difference rates between Huawei P20 Pro and Samsung A51 are 4.426, 4.355, and 18.596 for CC, NCC, and PCE at reference $n_{mle} = 10$. On the contrary, the thresholds are similar for a certain FITS or FITS⁺ between different device models. The difference rates are around 1 for FITS and FITS⁺.

It is worth noting that in Section VI-D, we have shown VIVO IQOO U1X is not affected by software noise. Traditional PCE and our methods all achieve an EER of 0. Therefore, the results were not presented in Table II.

F. Comparison Among Test Statistics

To compare FITS, FITS⁺ and conventional approaches, we conduct experiments with 30 Huawei P20 Pro smartphones, which introduce heavy software noise. 20 camera fingerprints

TABLE III
COMPARISON AMONG NINE TEST STATISTICS

Algorithm	EER	efficiency ratio	Compatible with	
			software noise	no software noise
CC	11.50%	0.05	no	yes
NCC	10.10%	0.14	no	yes
PCE	9.86%	1.00	no	yes
FITS_CC	5.33%	0.44	yes	no
FITS_NCC	4.68%	0.87	yes	no
FITS_PCE	4.85%	4.46	yes	no
FITS ⁺ _CC	5.69%	0.44	yes	yes
FITS ⁺ _NCC	5.35%	0.87	yes	yes
FITS ⁺ _PCE	5.00%	4.46	yes	yes

are extracted for each device. Each camera fingerprint is extracted from 10 images. Table III shows the comparison among three conventional and six new test statistics in three aspects: EER (equal error rate), efficiency ratio (i.e., computation time taken by a statistic over computation time taken by PCE), and compatibility with fingerprints with and without software noise. Each statistic is tested 100 times to derive the average performance. Different images are used for each test.

For conventional algorithms, in terms of efficiency ratio, $CC < NCC < PCE$ and CC is most time efficient. The PCE algorithm takes 0.463 seconds on average for fingerprint matching, serving as a baseline for comparison with other algorithms. In terms of EER, $CC > NCC > PCE$, which performs best.

For FITS and FITS⁺, in terms of efficiency ratio, $FITS^{(+)}_{CC} < FITS^{(+)}_{NCC} < FITS^{(+)}_{PCE}$. In terms of EER, the performance of FITS⁺_NCC and FITS⁺_PCE is similar, but slightly better than FITS⁺_CC. Considering both efficiency and accuracy, we recommend the FITS⁺_NCC algorithm for camera fingerprint matching.

Our algorithms address the fundamental and novel issue of camera fingerprint matching in the presence of software noise. Camera fingerprinting is applied in multiple domains. Besides account linking, it can also be used for copyright validation and identity authentication. Regardless of various domains, the common process always includes matching. Therefore, if the matching process consistently faces reliability issues, account linking will be inaccurate, copyright validation results will be unreliable, and identity authentication systems will have security vulnerabilities. Currently, software noise severely affects the reliability of matching, so addressing the software noise issue can aid the deployment of many related applications.

G. Impact of Camera Software App

The change of camera app or software version does not affect the matching results. Users may take photos with various camera apps and upload them to different accounts, but these two accounts can still be matched through camera fingerprinting. We use FITS⁺_CC, as discussed in Section V-E, as an example:

$$FITS^{+}_{CC}(\hat{\mathbf{K}}_Q, \hat{\mathbf{K}}_R) \approx \frac{\alpha_Q \alpha_R \mathbf{K}_{Qm} \odot \mathbf{K}_{Rm}}{\beta_Q \beta_Z \mathbf{S}_{Qm} \odot \mathbf{S}_{Zm} + C} + 1. \quad (50)$$

For instance, there are two suspicious accounts (A and B), and their photos are taken through different camera apps on the same

TABLE IV
AUC OF CONVENTIONAL APPROACHES AND FITS/FITS⁺ IN CROSS-APP CAMERA FINGERPRINT MATCHING

Algorithm	AUC with different n_{mle}		
	6	8	10
CC	1.000	1.000	1.000
NCC	1.000	1.000	1.000
PCE	1.000	1.000	1.000
FITS_CC	0.898	0.915	0.922
FITS_NCC	0.895	0.909	0.918
FITS_PCE	0.896	0.905	0.913
FITS ⁺ _CC	1.000	1.000	1.000
FITS ⁺ _NCC	1.000	1.000	1.000
FITS ⁺ _PCE	1.000	1.000	1.000

device. We should determine whether the two accounts belong to the same user by checking if their camera fingerprints match. (i) If the two camera fingerprints originate from different cameras, $\mathbf{K}_{Qm} \odot \mathbf{K}_{Rm}$ approaches 0, and FITS⁺_CC approaches 1. (ii) If the camera fingerprints originate from the same camera, $\mathbf{K}_{Qm}$ matches $\mathbf{K}_{Rm}$. Due to the difference in software noise between the image from account A ($\mathbf{S}_{Qm}$) and the images from account B ($\mathbf{S}_{Zm}$), the value of $\mathbf{S}_{Qm} \odot \mathbf{S}_{Zm}$ decreases, as there is less correlation between $\mathbf{S}_{Qm}$ and $\mathbf{S}_{Zm}$ compared to the scenario with the same camera app for both the account A and account B images. Therefore, FITS⁺_CC is larger than 1, which makes it easier to distinguish between matching and non-matching cases. As a result, changes in the camera software do not affect the matching result.

To validate the abovementioned claim that changes in the camera app do not impact camera fingerprint matching, we conduct cross-app fingerprint matching with both conventional methods and FITS (FITS⁺) on 30 Samsung A51 smartphones. The photos for account A are captured through the built-in camera app, while the photos for account B are taken with the third-party camera app B612.

The results are presented in Table IV. It is evident that conventional approaches yield very high AUC values, indicating that cross-app fingerprint matching in this scenario has degenerated to matching without software noise. In contrast, the FITS approaches do not perform as well as the conventional methods, as they are specifically designed for scenarios with the presence of similar software noise. As anticipated, the FITS⁺ approaches achieve high AUC scores because they are capable of handling all three scenarios—those with, without, and with different software noise. As for other scenarios like camera app/operating system updates, our method holds equally since they all result in software noise change.

VII. RELATED WORK

Camera fingerprinting mainly has two applications: (i) Fingerprint matching, which aims to detect which camera takes an image of interest or whether two images are taken by the same camera [18], [36], [37]; (ii) Forgery detection, which aims to detect forged images [38], [39], [40], [41] or locate which part of an image is tampered [26], [42].

The PRNU-based camera fingerprint is often extracted from images. There are many works improving the PRNU-based camera fingerprint technique from different perspectives. For better fingerprint matching, pre-processing and filter selection are refined in [43], [44], [45], [46], [47]. Robustness of camera fingerprint technique is investigated in complex contexts such as clipping and scaling images [48], high dynamic range (HDR) images [49], [50], and images under the influence of color filter array (CFA) [51]. The effectiveness of PRNU is studied for visual media on online social networks [16], [52], [53], [54]. Recently, data-driven strategies have been proposed to improve fingerprint matching efficiency and accuracy [30], [55], [56], [57]. Computation efficiency is studied when performing fingerprint matching in case of a large quantity of fingerprints [52], [57], [58], [59], [60], [61].

The camera fingerprinting can be applied to digital videos. Different from images, various encoding standards in video have significant impacts on the extracted fingerprint. Research has been conducted to suppress these impacts and improve the matching accuracy [53], [62], [63]. Robustness of video-based PRNU is researched in different scenarios, such as video blocking and blurring caused by packet loss during video transmission [64] and video stabilization during video capturing [29], [65], [66], [67]. There is also research to reduce the computation overhead in fingerprint extraction [68] and others aim to improve the matching accuracy by combining images and videos [68], [69].

Despite the active research on fingerprint matching, we are the first to study fingerprint matching in existence of software noise.

VIII. CONCLUSION

In this paper, we analyze account linking based on camera fingerprinting and study the impact of camera post-processing software-introduced noise. Then we propose novel test statistics for matching camera fingerprints polluted by software noise. The basic idea is to detect the correlated PRNU hardware noise component with our test statistics. For images from different cameras, since our test statistics always have a mean of around 1 for each camera, a common threshold can be used for fingerprint matching. Our FITS⁺ test statistics can handle images with and without software noise blindly. Extensive experiments are conducted to validate the effectiveness of our approaches. Compared with conventional approaches, our FITS and FITS⁺ test statistics achieve higher AUC efficiently, and are robust to various post-processing techniques such as digital watermarking. For example, under the watermarking technique—Patchwork, the AUC of FITS⁺_PCE is improved by 46.65% to 0.921 compared with the conventional PCE approach. ACL With FITS has the following advantages: 1. Robustness: It is not sensitive to the type and strength of software noise. 2. Unified Threshold: The normalized H0 distribution allows OSNs to set a uniform threshold, facilitating large-scale account linking. 3. High Stability: Even if an anonymous account uses any other camera app (with or without software noise), it cannot evade account linking. In summary, ACL provides an efficient solution

for account linking, helping to combat crimes related to online anonymity and enhancing the security of the online environment.

REFERENCES

- [1] D. Huntley, "County Durham paedophile who uploaded child abuse images to Facebook jailed," 2023. [Online]. Available: <https://www.chroniclive.co.uk/news/north-east-news/county-durham-paedophile-who-uploaded-27337608>
- [2] NCA, "Administrator of online forum for paedophiles jailed following NCA investigation," 2023. [Online]. Available: <https://www.nationalcrimeagency.gov.uk/news/administrator-of-online-forum-for-paedophiles-jailed-following-nca-investigation>
- [3] M. Lodgel, "Bondage-loving mother, 57, who ran Facebook fetish page faces jail after hosting child abuse images on her site," 2023. [Online]. Available: <https://www.dailymail.co.uk/news/article-1169153/Bondage-loving-mum-ran-Facebook-fetish-page-faces-jail-hosting-child-abuse-images-site.html>
- [4] AFP, "Queensland man jailed for sharing child abuse material," 2023. [Online]. Available: <https://www.afp.gov.au/news-media/media-releases/queensland-man-jailed-sharing-child-abuse-material>
- [5] J. Menn and D. Harwell, "Twitter under fire for reinstating account that posted child sex abuse," 2023. [Online]. Available: <https://www.washingtonpost.com/technology/2023/07/27/twitter-csam-dom-lucre-elon-musk/>
- [6] PhotoDNA, 2009. [Online]. Available: <https://www.microsoft.com/en-us/PhotoDNA>
- [7] U. Iqbal, S. Englehardt, and Z. Shafiq, "Fingerprinting the fingerprinters: Learning to detect browser fingerprinting behaviors," in *Proc. 2021 IEEE Symp. Secur. Privacy*, 2021, pp. 1143–1161.
- [8] F. Guo and T.-C. Chiueh, "Sequence number-based MAC address spoof detection," in *Proc. Int. Workshop Recent Adv. Intrusion Detection*, Springer, 2005, pp. 309–329.
- [9] A. Das, N. Borisov, and M. Caesar, "Tracking mobile web users through motion sensors: Attacks and defenses," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2016.
- [10] S. Dey, N. Roy, W. Xu, R. R. Choudhury, and S. Nelakuditi, "AccelPrint: Imperfections of accelerometers make smartphones trackable," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2014, pp. 23–26.
- [11] Z. Zhou, W. Diao, X. Liu, and K. Zhang, "Acoustic fingerprinting revisited: Generate stable device ID stealthily with inaudible sound," in *Proc. 2014 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2014, pp. 429–440.
- [12] A. Das, N. Borisov, and M. Caesar, "Do you hear what I hear? Fingerprinting smart devices through embedded acoustic components," in *Proc. 2014 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2014, pp. 441–452.
- [13] X. Zhou, X. Ji, C. Yan, J. Deng, and W. Xu, "NAuth: Secure face-to-face device authentication via nonlinearity," in *Proc. 2019 IEEE Conf. Comput. Commun.*, 2019, pp. 2080–2088.
- [14] Y. Cheng, X. Ji, J. Zhang, W. Xu, and Y.-C. Chen, "DeMiCPU: Device fingerprinting with magnetic signals radiated by CPU," in *Proc. 2019 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 1149–1170.
- [15] T. Laor et al., "DRAWNAPART: A device identification technique based on remote GPU fingerprinting," 2022, *arXiv:2201.09956*.
- [16] Z. Ba, S. Piao, X. Fu, D. Koutsonikolas, A. Mohaisen, and K. Ren, "ABC: Enabling smartphone authentication with built-in camera," in *Proc. 25th Annu. Netw. Distrib. Syst. Secur. Symp.*, 2018.
- [17] E. Altinisik, K. Tasdemir, and H. T. Sencar, "Mitigation of H.264 and H.265 video compression for reliable PRNU estimation," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 1557–1571, Oct. 2020.
- [18] J. Lukas, J. Fridrich, and M. Goljan, "Digital camera identification from sensor pattern noise," *IEEE Trans. Inf. Forensics Secur.*, vol. 1, no. 2, pp. 205–214, Jun. 2006.
- [19] F. Bertini, R. Sharma, A. Ianni, and D. Montesi, "Smartphone verification and user profiles linking across social networks by camera fingerprinting," in *Proc. 7th Int. Conf. Digit. Forensics Cyber Crime*, Seoul, South Korea, Springer, 2015, pp. 176–186.
- [20] R. Rouhi, F. Bertini, and D. Montesi, "A cluster-based approach of smartphone camera fingerprint for user profiles resolution within social network," in *Proc. 22nd Int. Database Eng. Appl. Symp.*, 2018, pp. 287–291.
- [21] R. Rahimeh, B. Flavio, and M. Danilo, "No matter what images you share, you can probably be fingerprinted anyway," *J. Imag.*, vol. 7, no. 2, 2021, Art. no. 33.

- [22] R. Satta, J. Galbally, and L. Beslay, "State-of-the-art review: Video analytics for fight against on-line child abuse," Inst. Protection Secur. Citizen, Eur. Commission-Joint Res. Centre, Ispra, VA, Italy, Tech. Rep. JRC85864, 2013.
- [23] F. Pathways, "New forensic image technology achieves success in major child abuse case," 2022. [Online]. Available: <https://www.forensic-pathways.com/new-forensic-image-technology-achieves-success-in-major-child-abuse-case/>
- [24] M. Goljan, J. Fridrich, and T. Filler, "Large scale test of sensor fingerprint camera identification," in *Proc. SPIE Media Forensics Secur.*, 2009, Art. no. 725401.
- [25] M. Chen, J. Fridrich, and M. Goljan, "Digital imaging sensor identification (further study)," in *Proc. SPIE Electron. Imag. Secur. Steganogr. Watermarking Multimedia Contents*, 2007, pp. 258–270.
- [26] M. Chen, J. Fridrich, M. Goljan, and J. Lukás, "Determining image origin and integrity using sensor noise," *IEEE Trans. Inf. Forensics Secur.*, vol. 3, no. 1, pp. 74–90, Mar. 2008.
- [27] C. Holt, "Two-channel likelihood detectors for arbitrary linear channel distortion," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 35, no. 3, pp. 267–273, Mar. 1987.
- [28] M. Goljan, "Digital camera identification from images—estimating false acceptance probability," in *Proc. Int. Workshop Digit. Watermarking*, Springer, 2008, pp. 454–468.
- [29] E. Altinisk and H. T. Sencar, "Source camera verification for strongly stabilized videos," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 643–657, Aug. 2021.
- [30] M. Fanfani, A. Piva, and C. Colombo, "PRNU registration under scale and rotation transform based on convolutional neural networks," *Pattern Recognit.*, vol. 124, 2022, Art. no. 108413.
- [31] B. Kaur, A. Kaur, and J. Singh, "Steganographic approach for hiding image in DCT domain," *Int. J. Adv. Eng. Technol.*, vol. 1, no. 3, 2011, Art. no. 72.
- [32] Z. Ba, X. Zhang, Z. Qin, and K. Ren, "CFP: Enabling camera fingerprint concealment for privacy-preserving image sharing," in *Proc. IEEE 39th Int. Conf. Distrib. Comput. Syst.*, 2019, pp. 1094–1105.
- [33] W. Bender, D. Gruhl, N. Morimoto, and A. Lu, "Techniques for data hiding," *IBM Syst. J.*, vol. 35, no. 3.4, pp. 313–336, 1996.
- [34] T. Counterpoint, "Global smartphone market share: By quarter," 2022. [Online]. Available: <https://www.counterpointresearch.com/global-smartphone-share/>
- [35] J. Belle Wong, "Top social media statistics and trends of 2023," 2023. [Online]. Available: <https://www.forbes.com/advisor/business/social-media-statistics/>
- [36] L. Liu, H. Yu, Z. Ba, L. Lu, F. Lin, and K. Ren, "PassFace: Enabling practical anti-spoofing facial recognition with camera fingerprinting," in *Proc. 2021 IEEE Int. Conf. Commun.*, Montreal, QC, Canada, 2021, pp. 1–6. [Online]. Available: <https://doi.org/10.1109/ICC42927.2021.9501053>
- [37] Z. Ba, Z. Qin, X. Fu, and K. Ren, "CIM: Camera in motion for smartphone authentication," *IEEE Trans. Inf. Forensics Secur.*, vol. 14, no. 11, pp. 2987–3002, Nov. 2019. [Online]. Available: <https://doi.org/10.1109/TIFS.2019.2911173>
- [38] G. Chierchia, G. Poggi, C. Sansone, and L. Verdoliva, "A Bayesian-MRF approach for PRNU-based image forgery detection," *IEEE Trans. Inf. Forensics Secur.*, vol. 9, no. 4, pp. 554–567, Apr. 2014.
- [39] M. Koopman, A. M. Rodriguez, and Z. Geradts, "Detection of deepfake video manipulation," in *Proc. 20th Ir. Mach. Vis. Image Process. Conf.*, 2018, pp. 133–136.
- [40] Y. Zheng, Y. Cao, and C.-H. Chang, "A PUF-based data-device hash for tampered image detection and source camera identification," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 620–634, Jul. 2020.
- [41] Y. Quan and C.-T. Li, "On addressing the impact of ISO speed upon PRNU and forgery detection," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 190–202, Jul. 2021.
- [42] P. Korus and J. Huang, "Multi-scale analysis strategies in PRNU-based tampering localization," *IEEE Trans. Inf. Forensics Secur.*, vol. 12, no. 4, pp. 809–824, Apr. 2017.
- [43] X. Kang, Y. Li, Z. Qu, and J. Huang, "Enhancing source camera identification performance with a camera reference phase sensor pattern noise," *IEEE Trans. Inf. Forensics Secur.*, vol. 7, no. 2, pp. 393–402, Apr. 2012.
- [44] X. Lin and C.-T. Li, "Preprocessing reference sensor pattern noise via spectrum equalization," *IEEE Trans. Inf. Forensics Secur.*, vol. 11, no. 1, pp. 126–140, Jan. 2016.
- [45] M. Al-Ani and F. Khelifi, "On the SPN estimation in image forensics: A systematic empirical evaluation," *IEEE Trans. Inf. Forensics Secur.*, vol. 12, no. 5, pp. 1067–1081, May 2017.
- [46] A. Lawgaly and F. Khelifi, "Sensor pattern noise estimation based on improved locally adaptive DCT filtering and weighted averaging for source camera identification and verification," *IEEE Trans. Inf. Forensics Secur.*, vol. 12, no. 2, pp. 392–404, Feb. 2017.
- [47] C.-T. Li, "Source camera identification using enhanced sensor pattern noise," *IEEE Trans. Inf. Forensics Secur.*, vol. 5, no. 2, pp. 280–287, Jun. 2010.
- [48] M. Goljan and J. Fridrich, "Camera identification from cropped and scaled images," in *Proc. SPIE Secur. Forensics Steganogr. Watermarking Multimedia Contents*, 2008, pp. 154–166.
- [49] O. Al Shaya, P. Yang, R. Ni, Y. Zhao, and A. Piva, "A new dataset for source identification of high dynamic range images," *Sensors*, vol. 18, no. 11, 2018, Art. no. 3801.
- [50] M. D. M. Hosseini and M. Goljan, "Camera identification from HDR images," in *Proc. ACM Workshop Inf. Hiding Multimedia Secur.*, 2019, pp. 69–76.
- [51] C.-T. Li and Y. Li, "Color-decoupled photo response non-uniformity for digital image forensics," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 22, no. 2, pp. 260–271, Feb. 2012.
- [52] S. Bayram, H. T. Sencar, and N. Memon, "Sensor fingerprint identification through composite fingerprints and group testing," *IEEE Trans. Inf. Forensics Secur.*, vol. 10, no. 3, pp. 597–612, Mar. 2015.
- [53] I. Amerini, R. Caldelli, A. Del Mastio, A. Di Fuccia, C. Molinari, and A. P. Rizzo, "Dealing with video source identification in social networks," *Signal Process. Image Commun.*, vol. 57, pp. 1–7, 2017.
- [54] F. Marra, G. Poggi, C. Sansone, and L. Verdoliva, "Blind PRNU-based image clustering for source identification," *IEEE Trans. Inf. Forensics Secur.*, vol. 12, no. 9, pp. 2197–2211, Sep. 2017.
- [55] P. Yang, R. Ni, Y. Zhao, and W. Zhao, "Source camera identification based on content-adaptive fusion residual networks," *Pattern Recognit. Lett.*, vol. 119, pp. 195–204, 2019.
- [56] S. Mandelli, D. Cozzolino, P. Bestagini, L. Verdoliva, and S. Tubaro, "CNN-based fast source device identification," *IEEE Signal Process. Lett.*, vol. 27, pp. 1285–1289, Jul. 2020.
- [57] A. Karunakar et al., "Identification of source social network of digital images using deep neural network," *Pattern Recognit. Lett.*, vol. 150, pp. 17–25, 2021.
- [58] S. Bayram, H. T. Sencar, and N. Memon, "Efficient sensor fingerprint matching through fingerprint binarization," *IEEE Trans. Inf. Forensics Secur.*, vol. 7, no. 4, pp. 1404–1413, Aug. 2012.
- [59] D. Valsesia, G. Coluccia, T. Bianchi, and E. Magli, "Compressed fingerprint matching and camera identification via random projections," *IEEE Trans. Inf. Forensics Secur.*, vol. 10, no. 7, pp. 1472–1485, Jul. 2015.
- [60] L. Bondi, P. Bestagini, F. Perez-Gonzalez, and S. Tubaro, "Improving PRNU compression through preprocessing, quantization, and coding," *IEEE Trans. Inf. Forensics Secur.*, vol. 14, no. 3, pp. 608–620, Mar. 2019.
- [61] R. Li, C.-T. Li, and Y. Guan, "Inference of a compact representation of sensor fingerprint for source camera identification," *Pattern Recognit.*, vol. 74, pp. 556–567, 2018.
- [62] R. R. López, A. L. S. Orozco, and L. J. G. Villalba, "Compression effects and scene details on the source camera identification of digital videos," *Expert Syst. Appl.*, vol. 170, 2021, Art. no. 114515.
- [63] E. Altinisk, K. Tasdemir, and H. T. Sencar, "Mitigation of H.264 and H.265 video compression for reliable PRNU estimation," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 1557–1571, Oct. 2020.
- [64] S. Chen, A. Pande, K. Zeng, and P. Mohapatra, "Live video forensics: Source identification in lossy wireless networks," *IEEE Trans. Inf. Forensics Secur.*, vol. 10, no. 1, pp. 28–39, Jan. 2015.
- [65] S. Mandelli, P. Bestagini, L. Verdoliva, and S. Tubaro, "Facing device attribution problem for stabilized video sequences," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 14–27, May 2020.
- [66] J. Li, Y. Lv, B. Ma, M. Yang, C. Wang, and Y. Zheng, "Video source identification algorithm based on 3D geometric transformation," *Comput. Syst. Sci. Eng.*, vol. 35, no. 6, pp. 513–521, 2020.
- [67] B. Ma, Y. Hu, J. Li, C. Wang, M. Yang, and Y. Zheng, "PRNU extraction from stabilized video: A patch maybe better than a bunch," *Comput. Syst. Sci. Eng.*, vol. 36, no. 1, pp. 189–200, 2021.
- [68] S. Taspinar, M. Mohanty, and N. Memon, "Camera identification of multi-format devices," *Pattern Recognit. Lett.*, vol. 140, pp. 288–294, 2020.
- [69] M. Iuliani, M. Fontani, D. Shullani, and A. Piva, "Hybrid reference-based video source identification," *Sensors*, vol. 19, no. 3, 2019, Art. no. 649.



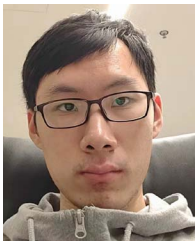
Liu Liu is currently working toward the PhD degree with the School of Cyber Science and Technology, Zhejiang University. His research interests lie in security and privacy aspects of Internet of Things and forensic analysis of multimedia content. He has authored or co-authored peer-reviewed papers published in ACM CCS, ACM Mobicom, and IEEE ICC.



Xinwen Fu (Senior Member, IEEE) is a professor with the Department of Computer Science, University of Massachusetts Lowell. He was a tenured associate professor with the Department of Computer Science, University of Central Florida. His current research interests include computer and network security and privacy. He has published at prestigious conferences including the four top computer security conferences (Oakland, CCS, USENIX Security and NDSS), and journals such as the *ACM/IEEE Transactions on Networking* (ToN) and *IEEE Transactions on Dependable and Secure Computing* (TDSC). He spoke at various technical security conferences including Black Hat.



Xiaodong Chen received the BS degree in information engineering from Zhejiang University, Hangzhou, China, in 2021. He is currently working toward the MS degree in cyberspace security with the School of Zhejiang University, Hangzhou. His research interests include IOT security.



Jianpeng Wang is currently working toward the PhD degree with the School of Cyber Science and Technology, Zhejiang University. His research interests lie in covert channel in Internet of Things and digital forensics. He has co-authored peer-reviewed papers published in ACM CCS.



Zhongjie Ba (Member, IEEE) received the PhD degree in computer science and engineering from the State University of New York at Buffalo, USA, in 2019. He is currently a professor with the School of Cyber Science and Technology, College of Computer Science and Technology, Zhejiang University, China. He was a postdoctoral researcher with the School of Computer Science, McGill University, Canada. His current research interests include the security and privacy aspects of Internet of Things, forensic analysis of multimedia content, and privacy-enhancing technologies in the context of collaborative deep learning. Results have been published in peer reviewed top conferences and journals, including CCS, S&P, NDSS, INFOCOM, WWW, ICDCS, and *IEEE Transactions on Information Forensics and Security*. Currently, he serves as an associate editor of the *IEEE Internet of Things Journal* and the technical program committee of several conferences in the field of Internet of Things and wireless communication.



Feng Lin (Senior Member, IEEE) received the PhD degree from the Department of Electrical and Computer Engineering, Tennessee Technological University, USA, in 2015. He is currently a professor with the School of Cyber Science and Technology, College of Computer Science and Technology, Zhejiang University, China. He was an assistant professor with the University of Colorado Denver, USA, a research scientist with the State University of New York (SUNY) at Buffalo, USA, and an engineer with Alcatel-Lucent (currently, Nokia). His current research interests include IoT security, smart vehicle security, AI security, and smart sensing. He was a recipient of the ACM SIGSAC China Rising Star Award, the Research Highlights Paper Award from ACM SIGMOBILE, Best Paper Awards from ACM MobiSys'20, IEEE Globecom'19, IEEE/ACM CHASE'22, IEEE BHI'17, the Best Demo Award from ACM HotMobile'18, and Best Paper Award Nominations from ACM SenSys'21 and IEEE INFOCOM'21. He serves as an editor of the *IEEE Network* and TPC chair of ACM Morse'22.



Li Lu (Member, IEEE) received the BE degree in computer science and technology from Xi'an Jiaotong University, and the PhD degree in computer science and technology from Shanghai Jiao Tong University. He is currently a tenure-track research professor with the School of Cyber Science and Technology and College of Computer Science and Technology, Zhejiang University. He was also a visiting research student with Wireless Information Network Laboratory (WINLAB) and Department of Electrical and Computer Engineering, Rutgers University. His research interests include IoT security, intelligent voice security, mobile sensing, and ubiquitous computing. He is the recipient of ACM China SIGAPP Chapter Rising Star Award, ACM China SIGAPP Chapter Doctoral Dissertation Award, Best Poster Runner-up Award from ACM MobiCom 2022 and First Runner-up Poster Award from ACM MobiCom 2019.



Kui Ren (Fellow, IEEE) received the PhD degree in electrical and computer engineering from Worcester Polytechnic Institute. He is professor and associate dean with the College of Computer Science and Technology, Zhejiang University, where he also directs the Institute of Cyber Science and Technology. Before that, he was SUNY empire innovation professor with the State University of New York at Buffalo, USA. His current research interests include data security, IoT security, AI security, and privacy. He received many recognitions including Guohua Distinguished Scholar Award of ZJU, IEEE CISTC Technical Recognition Award, SUNY Chancellor's Research Excellence Award, Sigma Xi Research Excellence Award, NSF CAREER Award, etc. He has published extensively in peer-reviewed journals and conferences and received the Test-of-time Paper Award from IEEE INFOCOM and many Best Paper Awards, including ACM MobiSys, IEEE ICDCS, IEEE ICNP, IEEE Globecom, ACM/IEEE IWQoS, etc. His h-index is 91, with a total citation exceeding 46,000 according to Google Scholar. He is a fellow of the ACM. He is a frequent reviewer for funding agencies internationally and serves on the editorial boards of many IEEE and ACM journals. Among others, he currently serves as chair of SIGSAC of ACM China Council, a member of ACM ASIACCS steering committee, and a member of S&T Committee of Ministry of Education of China.